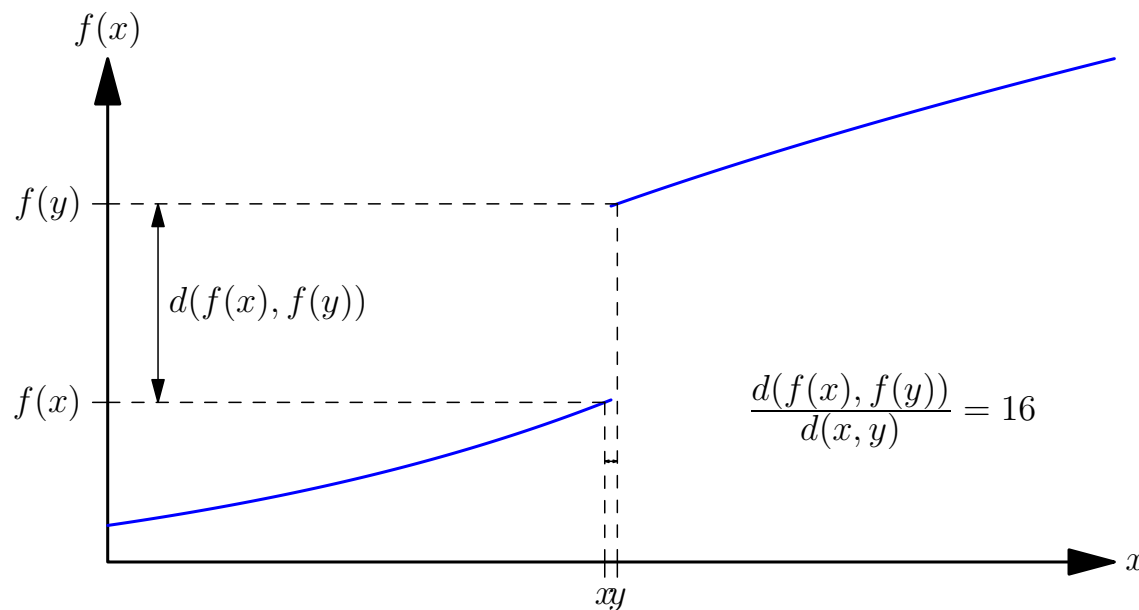


Mathematics of Machine Learning

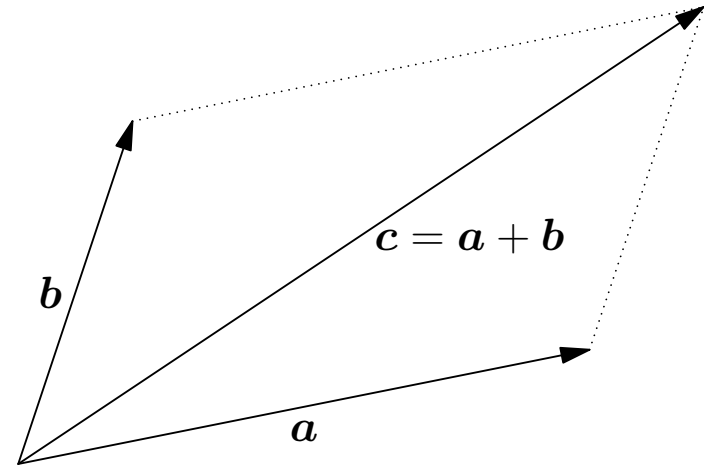
Vector Spaces



Vectors, metric spaces, norms

Outline

1. **Vector Spaces**
2. Metrics (distances)
3. Norms



Matrices, Vectors and All That

- The language of machine learning is mathematics
- Sometimes we draw pretty pictures to explain the mathematics
- Much of the mathematics we will use involves vectors, matrices and functions
- You need to master the language of mathematics, otherwise you won't understand the algorithms
- I'm going to spend this lecture and the next revising the mathematics you need to know

Matrices, Vectors and All That

- The language of machine learning is mathematics
- Sometimes we draw pretty pictures to explain the mathematics
- Much of the mathematics we will use involves vectors, matrices and functions
- You need to master the language of mathematics, otherwise you won't understand the algorithms
- I'm going to spend this lecture and the next revising the mathematics you need to know

Matrices, Vectors and All That

- The language of machine learning is mathematics
- Sometimes we draw pretty pictures to explain the mathematics
- Much of the mathematics we will use involves vectors, matrices and functions
- You need to master the language of mathematics, otherwise you won't understand the algorithms
- I'm going to spend this lecture and the next revising the mathematics you need to know

Matrices, Vectors and All That

- The language of machine learning is mathematics
- Sometimes we draw pretty pictures to explain the mathematics
- Much of the mathematics we will use involves vectors, matrices and functions
- You need to master the language of mathematics, otherwise you won't understand the algorithms
- I'm going to spend this lecture and the next revising the mathematics you need to know

Matrices, Vectors and All That

- The language of machine learning is mathematics
- Sometimes we draw pretty pictures to explain the mathematics
- Much of the mathematics we will use involves vectors, matrices and functions
- You need to master the language of mathematics, otherwise you won't understand the algorithms
- I'm going to spend this lecture and the next revising the mathematics you need to know

Matrices, Vectors and All That

- The language of machine learning is mathematics
- Sometimes we draw pretty pictures to explain the mathematics
- Much of the mathematics we will use involves vectors, matrices and functions
- You need to master the language of mathematics, otherwise you won't understand the algorithms
- I'm going to spend this lecture and the next revising the mathematics you need to know (but I'm going to use a slightly posher language than you are probably used to)

Scalars (Fields)

- Vector spaces involve **fields** (numbers)
- These are quantities we can add together ($a + b$) and multiply together ($a \times b$)

Scalars (Fields)

- Vector spaces involve **fields** (numbers)—aka **scalars**
- These are quantities we can add together ($a + b$) and multiply together ($a \times b$)

Scalars (Fields)

- Vector spaces involve **fields** (numbers)—aka **scalars**
- These are quantities we can add together ($a + b$) and multiply together ($a \times b$)

Scalars (Fields)

- Vector spaces involve **fields** (numbers)—aka **scalars**
- These are quantities we can add together ($a + b$) and multiply together ($a \times b$)
- Formally they form an Abelian group under addition with an identity 0

Scalars (Fields)

- Vector spaces involve **fields** (numbers)—aka **scalars**
- These are quantities we can add together ($a + b$) and multiply together ($a \times b$)
- Formally they form an Abelian group under addition with an identity 0 and excluding 0 an Abelian group under multiplication

Scalars (Fields)

- Vector spaces involve **fields** (numbers)—aka **scalars**
- These are quantities we can add together ($a + b$) and multiply together ($a \times b$)
- Formally they form an Abelian group under addition with an identity 0 and excluding 0 an Abelian group under multiplication and they are distributive

$$a \times (b + c) = a \times b + a \times c$$

Scalars (Fields)

- Vector spaces involve **fields** (numbers)—aka **scalars**
- These are quantities we can add together ($a + b$) and multiply together ($a \times b$)
- Formally they form an Abelian group under addition with an identity 0 and excluding 0 an Abelian group under multiplication and they are distributive

$$a \times (b + c) = a \times b + a \times c$$

- Although this sounds rather daunting don't panic

Scalars (Fields)

- Vector spaces involve **fields** (numbers)—aka **scalars**
- These are quantities we can add together ($a + b$) and multiply together ($a \times b$)
- Formally they form an Abelian group under addition with an identity 0 and excluding 0 an Abelian group under multiplication and they are distributive

$$a \times (b + c) = a \times b + a \times c$$

- Although this sounds rather daunting don't panic. They behave like numbers.

Scalars (Fields)

- Vector spaces involve **fields** (numbers)—aka **scalars**
- These are quantities we can add together ($a + b$) and multiply together ($a \times b$)
- Formally they form an Abelian group under addition with an identity 0 and excluding 0 an Abelian group under multiplication and they are distributive

$$a \times (b + c) = a \times b + a \times c$$

- Although this sounds rather daunting don't panic. They behave like numbers. The field might be rational numbers, reals, complex numbers or something a bit more exotic (not the integers: 2 has no integer inverse)

Scalars (Fields)

- Vector spaces involve **fields** (numbers)—aka **scalars**
- These are quantities we can add together ($a + b$) and multiply together ($a \times b$)
- Formally they form an Abelian group under addition with an identity 0 and excluding 0 an Abelian group under multiplication and they are distributive

$$a \times (b + c) = a \times b + a \times c$$

- Although this sounds rather daunting don't panic. They behave like numbers. The field might be rational numbers, reals, complex numbers or something a bit more exotic (not the integers: 2 has no integer inverse)—but we will almost always consider reals

Vectors

- We often work with objects with many components (features)
- To help handle this we will use vector notation
- We represent vectors by bold symbols
- All our vectors are column vectors by default
- We treat them as $n \times 1$ matrices
- We write row vectors as transposes of column vectors

$$\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}$$

$$\mathbf{y}^T = (y_1, y_2, \dots, y_n)$$

Vectors

- We often work with objects with many components (features)
- To help handle this we will use vector notation

$$\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}$$

- We represent vectors by bold symbols
 - All our vectors are column vectors by default
 - We treat them as $n \times 1$ matrices
- We write row vectors as transposes of column vectors

$$\mathbf{y}^T = (y_1, y_2, \dots, y_n)$$

Vectors

- We often work with objects with many components (features)
- To help handle this we will use vector notation
- We represent vectors by bold symbols
- All our vectors are column vectors by default
- We treat them as $n \times 1$ matrices
- We write row vectors as transposes of column vectors

$$\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}$$

$$\mathbf{y}^T = (y_1, y_2, \dots, y_n)$$

Vectors

- We often work with objects with many components (features)
- To help handle this we will use vector notation

$$\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}$$

- We represent vectors by bold symbols
 - All our vectors are column vectors by default
 - We treat them as $n \times 1$ matrices
- We write row vectors as transposes of column vectors

$$\mathbf{y}^T = (y_1, y_2, \dots, y_n)$$

Vectors

- We often work with objects with many components (features)
- To help handle this we will use vector notation
- We represent vectors by bold symbols
- All our vectors are column vectors by default
- We treat them as $n \times 1$ matrices
- We write row vectors as transposes of column vectors

$$\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}$$

$$\mathbf{y}^T = (y_1, y_2, \dots, y_n)$$

Vectors

- We often work with objects with many components (features)
- To help handle this we will use vector notation
- We represent vectors by bold symbols
- All our vectors are column vectors by default
- We treat them as $n \times 1$ matrices
- We write row vectors as transposes of column vectors

$$\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}$$

$$\mathbf{y}^T = (y_1, y_2, \dots, y_n)$$

Vectors

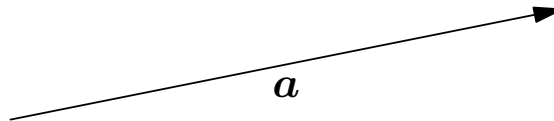
- We often work with objects with many components (features)
- To help handle this we will use vector notation
- We represent vectors by bold symbols
- All our vectors are column vectors by default
- We treat them as $n \times 1$ matrices
- We write row vectors as transposes of column vectors

$$\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix}$$

$$\mathbf{y}^T = (y_1, y_2, \dots, y_n)$$

Basic Vector Operations

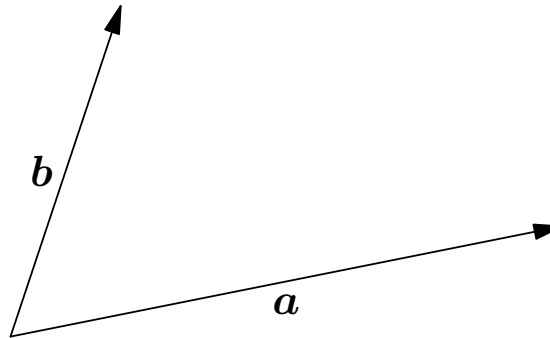
- The basic vector operations are adding



- multiplying by a scalar (a number)

Basic Vector Operations

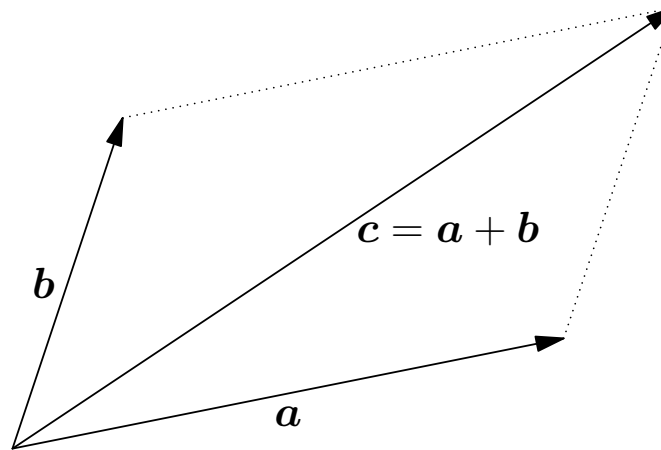
- The basic vector operations are adding



- multiplying by a scalar (a number)

Basic Vector Operations

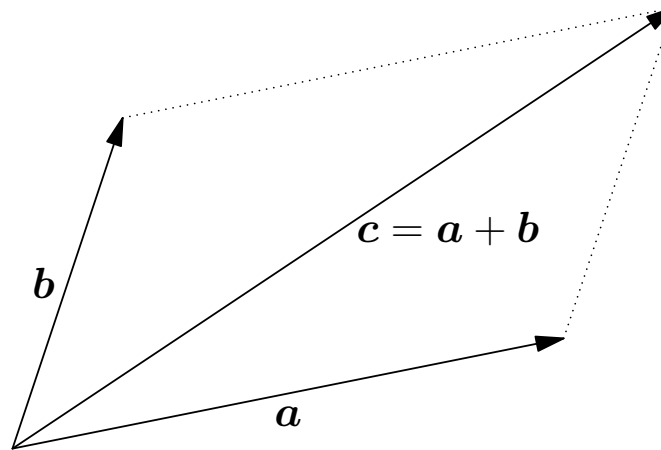
- The basic vector operations are adding



- multiplying by a scalar (a number)

Basic Vector Operations

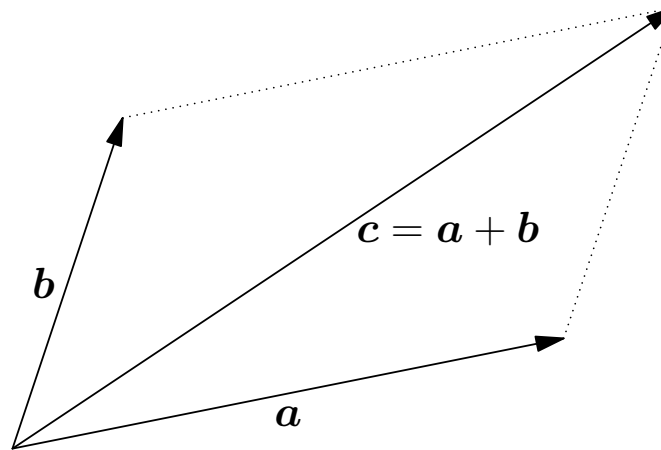
- The basic vector operations are adding



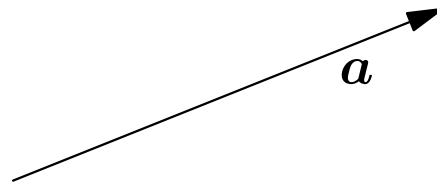
- multiplying by a scalar (a number)

Basic Vector Operations

- The basic vector operations are adding

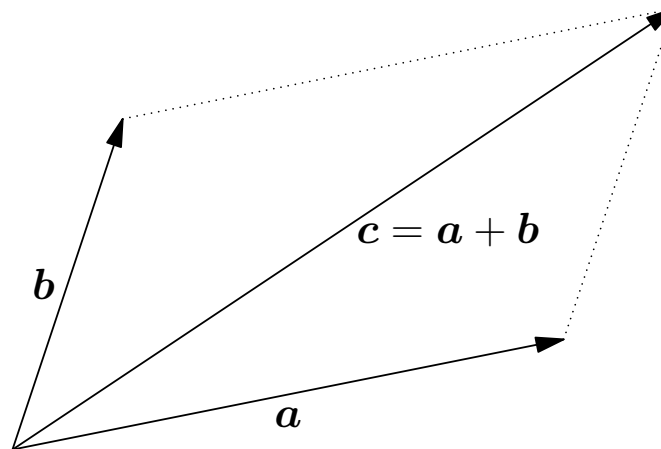


- multiplying by a scalar (a number)

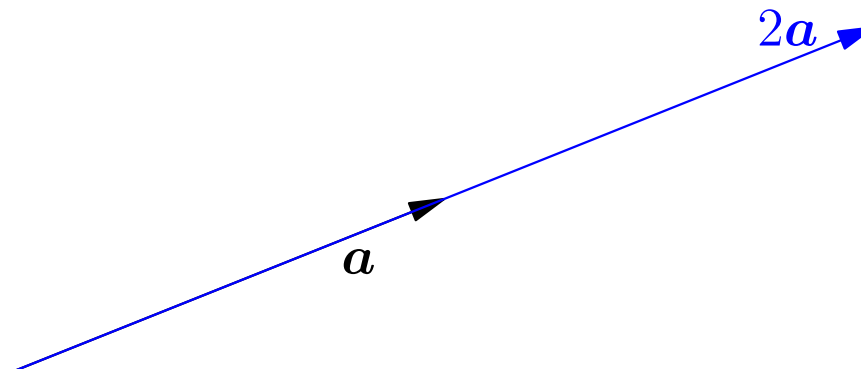


Basic Vector Operations

- The basic vector operations are adding



- multiplying by a scalar (a number)



Vector Space

- A vector space, $\mathcal{V}$, is a set of vectors which satisfies

1. if $\mathbf{v}, \mathbf{w} \in \mathcal{V}$ then $a\mathbf{v} \in \mathcal{V}$ and $\mathbf{v} + \mathbf{w} \in \mathcal{V}$ (closure)
2. $\mathbf{v} + \mathbf{w} = \mathbf{w} + \mathbf{v}$ (commutativity of addition)
3. $(\mathbf{u} + \mathbf{v}) + \mathbf{w} = \mathbf{u} + (\mathbf{v} + \mathbf{w})$ (associativity of addition)
4. $\mathbf{v} + \mathbf{0} = \mathbf{v}$ (existence of additive identity $\mathbf{0}$)
5. $\mathbf{v} + (-\mathbf{v}) = \mathbf{0}$ (existence of additive inverse $-\mathbf{v}$)
6. $1\mathbf{v} = \mathbf{v}$ (existence of multiplicative identity 1)
7. $a(b\mathbf{v}) = (ab)\mathbf{v}$ (associativity)
8. $a(\mathbf{v} + \mathbf{w}) = a\mathbf{v} + a\mathbf{w}$ (distributive properties)
9. $(a + b)\mathbf{v} = a\mathbf{v} + b\mathbf{v}$

(You don't need to remember these)

- Just from these properties we can deduce other properties

Vector Space

- A vector space, $\mathcal{V}$, is a set of vectors which satisfies
 1. if $\mathbf{v}, \mathbf{w} \in \mathcal{V}$ then $a\mathbf{v} \in \mathcal{V}$ and $\mathbf{v} + \mathbf{w} \in \mathcal{V}$ (closure)
 2. $\mathbf{v} + \mathbf{w} = \mathbf{w} + \mathbf{v}$ (commutativity of addition)
 3. $(\mathbf{u} + \mathbf{v}) + \mathbf{w} = \mathbf{u} + (\mathbf{v} + \mathbf{w})$ (associativity of addition)
 4. $\mathbf{v} + \mathbf{0} = \mathbf{v}$ (existence of additive identity $\mathbf{0}$)
 5. $\mathbf{v} + (-\mathbf{v}) = \mathbf{0}$ (existence of additive inverse $-\mathbf{v}$)
 6. $1\mathbf{v} = \mathbf{v}$ (existence of multiplicative identity 1)
 7. $a(b\mathbf{v}) = (ab)\mathbf{v}$ (associativity)
 8. $a(\mathbf{v} + \mathbf{w}) = a\mathbf{v} + a\mathbf{w}$ (distributive properties)
 9. $(a + b)\mathbf{v} = a\mathbf{v} + b\mathbf{v}$

(You don't need to remember these)

- Just from these properties we can deduce other properties

Vector Space

- A vector space, $\mathcal{V}$, is a set of vectors which satisfies
 1. if $\mathbf{v}, \mathbf{w} \in \mathcal{V}$ then $a\mathbf{v} \in \mathcal{V}$ and $\mathbf{v} + \mathbf{w} \in \mathcal{V}$ (closure)
 2. $\mathbf{v} + \mathbf{w} = \mathbf{w} + \mathbf{v}$ (commutativity of addition)
 3. $(\mathbf{u} + \mathbf{v}) + \mathbf{w} = \mathbf{u} + (\mathbf{v} + \mathbf{w})$ (associativity of addition)
 4. $\mathbf{v} + \mathbf{0} = \mathbf{v}$ (existence of additive identity $\mathbf{0}$)
 5. $\mathbf{v} + (-\mathbf{v}) = \mathbf{0}$ (existence of additive inverse $-\mathbf{v}$)
 6. $1\mathbf{v} = \mathbf{v}$ (existence of multiplicative identity 1)
 7. $a(b\mathbf{v}) = (ab)\mathbf{v}$ (associativity)
 8. $a(\mathbf{v} + \mathbf{w}) = a\mathbf{v} + a\mathbf{w}$ (distributive properties)
 9. $(a + b)\mathbf{v} = a\mathbf{v} + b\mathbf{v}$

(You don't need to remember these)

- Just from these properties we can deduce other properties

Vector Space

- A vector space, $\mathcal{V}$, is a set of vectors which satisfies
 1. if $\mathbf{v}, \mathbf{w} \in \mathcal{V}$ then $a\mathbf{v} \in \mathcal{V}$ and $\mathbf{v} + \mathbf{w} \in \mathcal{V}$ (closure)
 2. $\mathbf{v} + \mathbf{w} = \mathbf{w} + \mathbf{v}$ (commutativity of addition)
 3. $(\mathbf{u} + \mathbf{v}) + \mathbf{w} = \mathbf{u} + (\mathbf{v} + \mathbf{w})$ (associativity of addition)
 4. $\mathbf{v} + \mathbf{0} = \mathbf{v}$ (existence of additive identity $\mathbf{0}$)
 5. $\mathbf{v} + (-\mathbf{v}) = \mathbf{0}$ (existence of additive inverse $-\mathbf{v}$)
 6. $1\mathbf{v} = \mathbf{v}$ (existence of multiplicative identity 1)
 7. $a(b\mathbf{v}) = (ab)\mathbf{v}$ (associativity)
 8. $a(\mathbf{v} + \mathbf{w}) = a\mathbf{v} + a\mathbf{w}$ (distributive properties)
 9. $(a + b)\mathbf{v} = a\mathbf{v} + b\mathbf{v}$

(You don't need to remember these)

- Just from these properties we can deduce other properties

Vector Space

- A vector space, $\mathcal{V}$, is a set of vectors which satisfies
 1. if $\mathbf{v}, \mathbf{w} \in \mathcal{V}$ then $a\mathbf{v} \in \mathcal{V}$ and $\mathbf{v} + \mathbf{w} \in \mathcal{V}$ (closure)
 2. $\mathbf{v} + \mathbf{w} = \mathbf{w} + \mathbf{v}$ (commutativity of addition)
 3. $(\mathbf{u} + \mathbf{v}) + \mathbf{w} = \mathbf{u} + (\mathbf{v} + \mathbf{w})$ (associativity of addition)
 4. $\mathbf{v} + \mathbf{0} = \mathbf{v}$ (existence of additive identity $\mathbf{0}$)
 5. $\mathbf{v} + (-\mathbf{v}) = \mathbf{0}$ (existence of additive inverse $-\mathbf{v}$)
 6. $1\mathbf{v} = \mathbf{v}$ (existence of multiplicative identity 1)
 7. $a(b\mathbf{v}) = (ab)\mathbf{v}$ (associativity)
 8. $a(\mathbf{v} + \mathbf{w}) = a\mathbf{v} + a\mathbf{w}$ (distributive properties)
 9. $(a + b)\mathbf{v} = a\mathbf{v} + b\mathbf{v}$

(You don't need to remember these)

- Just from these properties we can deduce other properties

Vector Space

- A vector space, $\mathcal{V}$, is a set of vectors which satisfies
 1. if $\mathbf{v}, \mathbf{w} \in \mathcal{V}$ then $a\mathbf{v} \in \mathcal{V}$ and $\mathbf{v} + \mathbf{w} \in \mathcal{V}$ (closure)
 2. $\mathbf{v} + \mathbf{w} = \mathbf{w} + \mathbf{v}$ (commutativity of addition)
 3. $(\mathbf{u} + \mathbf{v}) + \mathbf{w} = \mathbf{u} + (\mathbf{v} + \mathbf{w})$ (associativity of addition)
 4. $\mathbf{v} + \mathbf{0} = \mathbf{v}$ (existence of additive identity $\mathbf{0}$)
 5. $\mathbf{v} + (-\mathbf{v}) = \mathbf{0}$ (existence of additive inverse $-\mathbf{v}$)
 6. $1\mathbf{v} = \mathbf{v}$ (existence of multiplicative identity 1)
 7. $a(b\mathbf{v}) = (ab)\mathbf{v}$ (associativity)
 8. $a(\mathbf{v} + \mathbf{w}) = a\mathbf{v} + a\mathbf{w}$ (distributive properties)
 9. $(a + b)\mathbf{v} = a\mathbf{v} + b\mathbf{v}$

(You don't need to remember these)

- Just from these properties we can deduce other properties

Vector Space

- A vector space, $\mathcal{V}$, is a set of vectors which satisfies
 1. if $\mathbf{v}, \mathbf{w} \in \mathcal{V}$ then $a\mathbf{v} \in \mathcal{V}$ and $\mathbf{v} + \mathbf{w} \in \mathcal{V}$ (closure)
 2. $\mathbf{v} + \mathbf{w} = \mathbf{w} + \mathbf{v}$ (commutativity of addition)
 3. $(\mathbf{u} + \mathbf{v}) + \mathbf{w} = \mathbf{u} + (\mathbf{v} + \mathbf{w})$ (associativity of addition)
 4. $\mathbf{v} + \mathbf{0} = \mathbf{v}$ (existence of additive identity $\mathbf{0}$)
 5. $\mathbf{v} + (-\mathbf{v}) = \mathbf{0}$ (existence of additive inverse $-\mathbf{v}$)
 6. $1\mathbf{v} = \mathbf{v}$ (existence of multiplicative identity 1)
 7. $a(b\mathbf{v}) = (ab)\mathbf{v}$ (associativity)
 8. $a(\mathbf{v} + \mathbf{w}) = a\mathbf{v} + a\mathbf{w}$ (distributive properties)
 9. $(a + b)\mathbf{v} = a\mathbf{v} + b\mathbf{v}$

(You don't need to remember these)

- Just from these properties we can deduce other properties

Vector Space

- A vector space, $\mathcal{V}$, is a set of vectors which satisfies
 1. if $\mathbf{v}, \mathbf{w} \in \mathcal{V}$ then $a\mathbf{v} \in \mathcal{V}$ and $\mathbf{v} + \mathbf{w} \in \mathcal{V}$ (closure)
 2. $\mathbf{v} + \mathbf{w} = \mathbf{w} + \mathbf{v}$ (commutativity of addition)
 3. $(\mathbf{u} + \mathbf{v}) + \mathbf{w} = \mathbf{u} + (\mathbf{v} + \mathbf{w})$ (associativity of addition)
 4. $\mathbf{v} + \mathbf{0} = \mathbf{v}$ (existence of additive identity $\mathbf{0}$)
 5. $\mathbf{v} + (-\mathbf{v}) = \mathbf{0}$ (existence of additive inverse $-\mathbf{v}$)
 6. $1\mathbf{v} = \mathbf{v}$ (existence of multiplicative identity 1)
 7. $a(b\mathbf{v}) = (ab)\mathbf{v}$ (associativity)
 8. $a(\mathbf{v} + \mathbf{w}) = a\mathbf{v} + a\mathbf{w}$ (distributive properties)
 9. $(a + b)\mathbf{v} = a\mathbf{v} + b\mathbf{v}$

(You don't need to remember these)

- Just from these properties we can deduce other properties

Vector Space

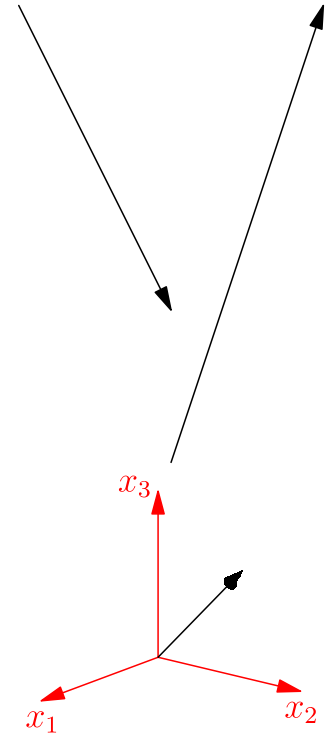
- A vector space, $\mathcal{V}$, is a set of vectors which satisfies
 1. if $\mathbf{v}, \mathbf{w} \in \mathcal{V}$ then $a\mathbf{v} \in \mathcal{V}$ and $\mathbf{v} + \mathbf{w} \in \mathcal{V}$ (closure)
 2. $\mathbf{v} + \mathbf{w} = \mathbf{w} + \mathbf{v}$ (commutativity of addition)
 3. $(\mathbf{u} + \mathbf{v}) + \mathbf{w} = \mathbf{u} + (\mathbf{v} + \mathbf{w})$ (associativity of addition)
 4. $\mathbf{v} + \mathbf{0} = \mathbf{v}$ (existence of additive identity $\mathbf{0}$)
 5. $\mathbf{v} + (-\mathbf{v}) = \mathbf{0}$ (existence of additive inverse $-\mathbf{v}$)
 6. $1\mathbf{v} = \mathbf{v}$ (existence of multiplicative identity 1)
 7. $a(b\mathbf{v}) = (ab)\mathbf{v}$ (associativity)
 8. $a(\mathbf{v} + \mathbf{w}) = a\mathbf{v} + a\mathbf{w}$ (distributive properties)
 9. $(a + b)\mathbf{v} = a\mathbf{v} + b\mathbf{v}$

(You don't need to remember these)

- Just from these properties we can deduce other properties

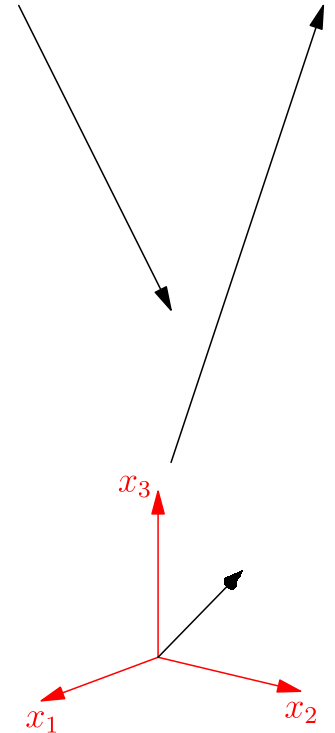
$$\mathbb{R}^n$$

- When we first learn about vectors we think of them as arrows in 3-D space
- If we centre them all at the origin then there is a one-to-one correspondence between vectors and points in space
- We call this vector space $\mathbb{R}^3$
- Any set of quantities $\mathbf{x} = (x_1, x_2, \dots, x_n)^\top$ which satisfy the axioms above form a vector space $\mathbb{R}^n$
- Of course, we can't so easily draw pictures of high-dimensional vectors



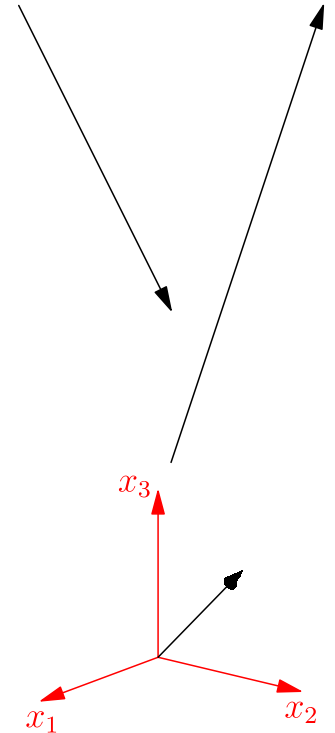
$$\mathbb{R}^n$$

- When we first learn about vectors we think of them as arrows in 3-D space
- If we centre them all at the origin then there is a one-to-one correspondence between vectors and points in space
- We call this vector space $\mathbb{R}^3$
- Any set of quantities $\mathbf{x} = (x_1, x_2, \dots, x_n)^\top$ which satisfy the axioms above form a vector space $\mathbb{R}^n$
- Of course, we can't so easily draw pictures of high-dimensional vectors



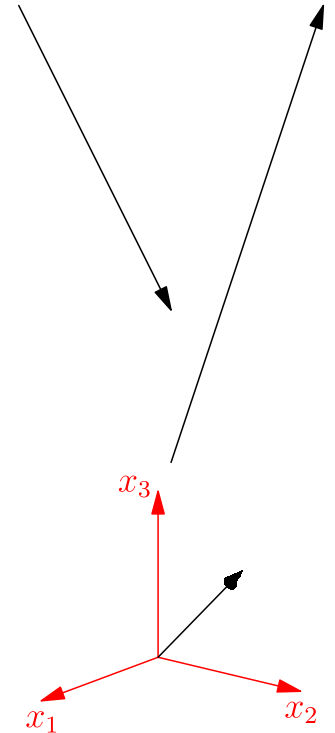
$$\mathbb{R}^n$$

- When we first learn about vectors we think of them as arrows in 3-D space
- If we centre them all at the origin then there is a one-to-one correspondence between vectors and points in space
- We call this vector space $\mathbb{R}^3$
- Any set of quantities $\mathbf{x} = (x_1, x_2, \dots, x_n)^\top$ which satisfy the axioms above form a vector space $\mathbb{R}^n$
- Of course, we can't so easily draw pictures of high-dimensional vectors



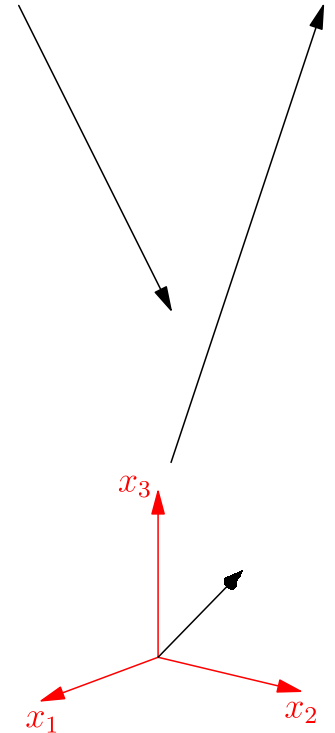
$$\mathbb{R}^n$$

- When we first learn about vectors we think of them as arrows in 3-D space
- If we centre them all at the origin then there is a one-to-one correspondence between vectors and points in space
- We call this vector space $\mathbb{R}^3$
- Any set of quantities $\mathbf{x} = (x_1, x_2, \dots, x_n)^\top$ which satisfy the axioms above form a vector space $\mathbb{R}^n$
- Of course, we can't so easily draw pictures of high-dimensional vectors



$$\mathbb{R}^n$$

- When we first learn about vectors we think of them as arrows in 3-D space
- If we centre them all at the origin then there is a one-to-one correspondence between vectors and points in space
- We call this vector space $\mathbb{R}^3$
- Any set of quantities $\mathbf{x} = (x_1, x_2, \dots, x_n)^\top$ which satisfy the axioms above form a vector space $\mathbb{R}^n$
- Of course, we can't so easily draw pictures of high-dimensional vectors



Other Vector Spaces

- Any objects that satisfy the axioms of a vector space are *vectors*
- Matrices satisfy all the conditions of a vector space
- Infinite sequences form a vector space
- Functions form a vector space
 - ★ Let $C(a,b)$ be the set of functions defined on the interval $[a,b]$
 - ★ Note that if $f(x), g(x) \in C(a,b)$ then $\alpha f(x) \in C(a,b)$ and $f(x) + g(x) \in C(a,b)$
- Bounded vectors in $\mathbb{R}^n$ **don't** form a vector space

Other Vector Spaces

- Any objects that satisfy the axioms of a vector space are *vectors*—not just $v \in \mathbb{R}^n$
- Matrices satisfy all the conditions of a vector space
- Infinite sequences form a vector space
- Functions form a vector space
 - ★ Let $C(a,b)$ be the set of functions defined on the interval $[a,b]$
 - ★ Note that if $f(x), g(x) \in C(a,b)$ then $\alpha f(x) \in C(a,b)$ and $f(x) + g(x) \in C(a,b)$
- Bounded vectors in $\mathbb{R}^n$ **don't** form a vector space

Other Vector Spaces

- Any objects that satisfy the axioms of a vector space are *vectors*—not just $v \in \mathbb{R}^n$
- Matrices satisfy all the conditions of a vector space
- Infinite sequences form a vector space
- Functions form a vector space
 - ★ Let $C(a,b)$ be the set of functions defined on the interval $[a,b]$
 - ★ Note that if $f(x), g(x) \in C(a,b)$ then $\alpha f(x) \in C(a,b)$ and $f(x) + g(x) \in C(a,b)$
- Bounded vectors in $\mathbb{R}^n$ **don't** form a vector space

Other Vector Spaces

- Any objects that satisfy the axioms of a vector space are *vectors*—not just $v \in \mathbb{R}^n$
- Matrices satisfy all the conditions of a vector space
- Infinite sequences form a vector space
- Functions form a vector space
 - ★ Let $C(a,b)$ be the set of functions defined on the interval $[a,b]$
 - ★ Note that if $f(x), g(x) \in C(a,b)$ then $\alpha f(x) \in C(a,b)$ and $f(x) + g(x) \in C(a,b)$
- Bounded vectors in $\mathbb{R}^n$ **don't** form a vector space

Other Vector Spaces

- Any objects that satisfy the axioms of a vector space are *vectors*—not just $v \in \mathbb{R}^n$
- Matrices satisfy all the conditions of a vector space
- Infinite sequences form a vector space
- Functions form a vector space
 - ★ Let $C(a,b)$ be the set of functions defined on the interval $[a,b]$
 - ★ Note that if $f(x), g(x) \in C(a,b)$ then $\alpha f(x) \in C(a,b)$ and $f(x) + g(x) \in C(a,b)$
- Bounded vectors in $\mathbb{R}^n$ **don't** form a vector space

Other Vector Spaces

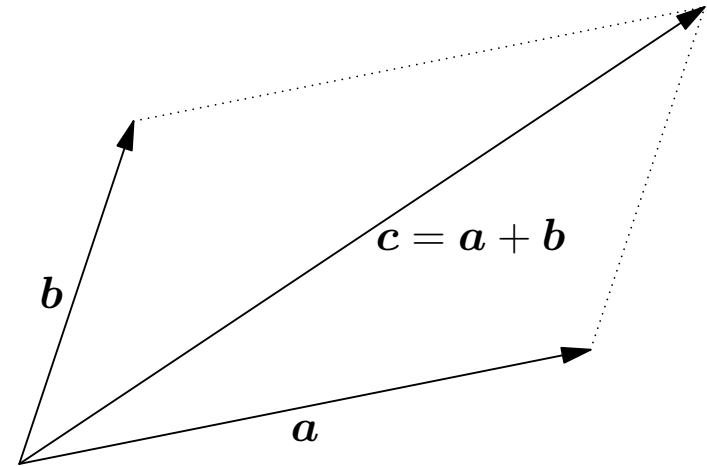
- Any objects that satisfy the axioms of a vector space are *vectors*—not just $v \in \mathbb{R}^n$
- Matrices satisfy all the conditions of a vector space
- Infinite sequences form a vector space
- Functions form a vector space
 - ★ Let $C(a,b)$ be the set of functions defined on the interval $[a,b]$
 - ★ Note that if $f(x), g(x) \in C(a,b)$ then $\alpha f(x) \in C(a,b)$ and $f(x) + g(x) \in C(a,b)$
- Bounded vectors in $\mathbb{R}^n$ **don't** form a vector space

Other Vector Spaces

- Any objects that satisfy the axioms of a vector space are *vectors*—not just $v \in \mathbb{R}^n$
- Matrices satisfy all the conditions of a vector space
- Infinite sequences form a vector space
- Functions form a vector space
 - ★ Let $C(a,b)$ be the set of functions defined on the interval $[a,b]$
 - ★ Note that if $f(x), g(x) \in C(a,b)$ then $\alpha f(x) \in C(a,b)$ and $f(x) + g(x) \in C(a,b)$
- Bounded vectors in $\mathbb{R}^n$ **don't** form a vector space

Outline

1. Vector Spaces
2. **Metrics (distances)**
3. Norms



Metrics

- Vector spaces become more interesting if we have a notion of distance
- We say $d(\mathbf{x}, \mathbf{y})$ is a **distance** or **metric** if
 1. $d(\mathbf{x}, \mathbf{y}) \geq 0$ (non-negativity)
 2. $d(\mathbf{x}, \mathbf{y}) = 0$ iff $\mathbf{x} = \mathbf{y}$ (identity of indiscernibles)
 3. $d(\mathbf{x}, \mathbf{y}) = d(\mathbf{y}, \mathbf{x})$ (symmetry)
 4. $d(\mathbf{x}, \mathbf{y}) \leq d(\mathbf{x}, \mathbf{z}) + d(\mathbf{z}, \mathbf{y})$ (triangular inequality)
- There are typically many possible distances (e.g. Euclidean distance, Manhattan distance, etc.)
- If one or more of the conditions aren't satisfied we have a **pseudo-metric**

Metrics

- Vector spaces become more interesting if we have a notion of distance
- We say $d(x, y)$ is a **distance** or **metric** if
 1. $d(x, y) \geq 0$ (non-negativity)
 2. $d(x, y) = 0$ iff $x = y$ (identity of indiscernibles)
 3. $d(x, y) = d(y, x)$ (symmetry)
 4. $d(x, y) \leq d(x, z) + d(z, y)$ (triangular inequality)
- There are typically many possible distances (e.g. Euclidean distance, Manhattan distance, etc.)
- If one or more of the conditions aren't satisfied we have a **pseudo-metric**

Metrics

- Vector spaces become more interesting if we have a notion of distance
- We say $d(\mathbf{x}, \mathbf{y})$ is a **distance** or **metric** if
 1. $d(\mathbf{x}, \mathbf{y}) \geq 0$ (non-negativity)
 2. $d(\mathbf{x}, \mathbf{y}) = 0$ iff $\mathbf{x} = \mathbf{y}$ (identity of indiscernibles)
 3. $d(\mathbf{x}, \mathbf{y}) = d(\mathbf{y}, \mathbf{x})$ (symmetry)
 4. $d(\mathbf{x}, \mathbf{y}) \leq d(\mathbf{x}, \mathbf{z}) + d(\mathbf{z}, \mathbf{y})$ (triangular inequality)
- There are typically many possible distances (e.g. Euclidean distance, Manhattan distance, etc.)
- If one or more of the conditions aren't satisfied we have a **pseudo-metric**

Metrics

- Vector spaces become more interesting if we have a notion of distance
- We say $d(x, y)$ is a **distance** or **metric** if
 1. $d(x, y) \geq 0$ (non-negativity)
 2. $d(x, y) = 0$ iff $x = y$ (identity of indiscernibles)
 3. $d(x, y) = d(y, x)$ (symmetry)
 4. $d(x, y) \leq d(x, z) + d(z, y)$ (triangular inequality)
- There are typically many possible distances (e.g. Euclidean distance, Manhattan distance, etc.)
- If one or more of the conditions aren't satisfied we have a **pseudo-metric**

Metrics

- Vector spaces become more interesting if we have a notion of distance
- We say $d(\mathbf{x}, \mathbf{y})$ is a **distance** or **metric** if
 1. $d(\mathbf{x}, \mathbf{y}) \geq 0$ (non-negativity)
 2. $d(\mathbf{x}, \mathbf{y}) = 0$ iff $\mathbf{x} = \mathbf{y}$ (identity of indiscernibles)
 3. $d(\mathbf{x}, \mathbf{y}) = d(\mathbf{y}, \mathbf{x})$ (symmetry)
 4. $d(\mathbf{x}, \mathbf{y}) \leq d(\mathbf{x}, \mathbf{z}) + d(\mathbf{z}, \mathbf{y})$ (triangular inequality)
- There are typically many possible distances (e.g. Euclidean distance, Manhattan distance, etc.)
- If one or more of the conditions aren't satisfied we have a **pseudo-metric**

Metrics

- Vector spaces become more interesting if we have a notion of distance
- We say $d(\mathbf{x}, \mathbf{y})$ is a **distance** or **metric** if
 1. $d(\mathbf{x}, \mathbf{y}) \geq 0$ (non-negativity)
 2. $d(\mathbf{x}, \mathbf{y}) = 0$ iff $\mathbf{x} = \mathbf{y}$ (identity of indiscernibles)
 3. $d(\mathbf{x}, \mathbf{y}) = d(\mathbf{y}, \mathbf{x})$ (symmetry)
 4. $d(\mathbf{x}, \mathbf{y}) \leq d(\mathbf{x}, \mathbf{z}) + d(\mathbf{z}, \mathbf{y})$ (triangular inequality)
- There are typically many possible distances (e.g. Euclidean distance, Manhattan distance, etc.)
- If one or more of the conditions aren't satisfied we have a **pseudo-metric**

Metrics

- Vector spaces become more interesting if we have a notion of distance
- We say $d(\mathbf{x}, \mathbf{y})$ is a **distance** or **metric** if
 1. $d(\mathbf{x}, \mathbf{y}) \geq 0$ (non-negativity)
 2. $d(\mathbf{x}, \mathbf{y}) = 0$ iff $\mathbf{x} = \mathbf{y}$ (identity of indiscernibles)
 3. $d(\mathbf{x}, \mathbf{y}) = d(\mathbf{y}, \mathbf{x})$ (symmetry)
 4. $d(\mathbf{x}, \mathbf{y}) \leq d(\mathbf{x}, \mathbf{z}) + d(\mathbf{z}, \mathbf{y})$ (triangular inequality)
- There are typically many possible distances (e.g. Euclidean distance, Manhattan distance, etc.)
- If one or more of the conditions aren't satisfied we have a **pseudo-metric**

Mappings and Functions

- A function defines a mapping from one vector space to another (although the spaces might be the same)

Mappings and Functions

- A function defines a mapping from one vector space to another (although the spaces might be the same), e.g.

$$f : \mathbb{R} \rightarrow \mathbb{R}$$

Mappings and Functions

- A function defines a mapping from one vector space to another (although the spaces might be the same), e.g.

$$f : \mathbb{R} \rightarrow \mathbb{R}$$

(f maps the reals to the reals, i.e. $f(x)$ takes a real x and gives you a new real number $y = f(x)$)

Mappings and Functions

- A function defines a mapping from one vector space to another (although the spaces might be the same), e.g.

$$f : \mathbb{R} \rightarrow \mathbb{R}$$

(f maps the reals to the reals, i.e. $f(x)$ takes a real x and gives you a new real number $y = f(x)$)

- We are often interested in functions that behave nicely

Mappings and Functions

- A function defines a mapping from one vector space to another (although the spaces might be the same), e.g.

$$f : \mathbb{R} \rightarrow \mathbb{R}$$

(f maps the reals to the reals, i.e. $f(x)$ takes a real x and gives you a new real number $y = f(x)$)

- We are often interested in functions that behave nicely
- E.g. They are continuous

Lipschitz Function

- One way to characterise a well-behaved function, $f(x)$, is if there exists a number $K < \infty$ such that for all x and y

$$d(f(x), f(y)) \leq K d(x, y)$$

- This is known as a **Lipschitz condition** and the function is said to be K -Lipschitz
- Note that such functions cannot have any jumps (i.e. they are continuous)
- The size of K measures the limit on the amplifying effect of the function

Lipschitz Function

- One way to characterise a well-behaved function, $f(x)$, is if there exists a number $K < \infty$ such that for all x and y

$$d(f(x), f(y)) \leq K d(x, y)$$

- This is known as a **Lipschitz condition** and the function is said to be K -Lipschitz
- Note that such functions cannot have any jumps (i.e. they are continuous)
- The size of K measures the limit on the amplifying effect of the function

Lipschitz Function

- One way to characterise a well-behaved function, $f(x)$, is if there exists a number $K < \infty$ such that for all x and y

$$d(f(x), f(y)) \leq K d(x, y)$$

- This is known as a **Lipschitz condition** and the function is said to be K -Lipschitz or Lipschitz continuous
- Note that such functions cannot have any jumps (i.e. they are continuous)
- The size of K measures the limit on the amplifying effect of the function

Lipschitz Function

- One way to characterise a well-behaved function, $f(x)$, is if there exists a number $K < \infty$ such that for all x and y

$$d(f(x), f(y)) \leq K d(x, y)$$

- This is known as a **Lipschitz condition** and the function is said to be K -Lipschitz or Lipschitz continuous
- Note that such functions cannot have any jumps (i.e. they are continuous)
- The size of K measures the limit on the amplifying effect of the function

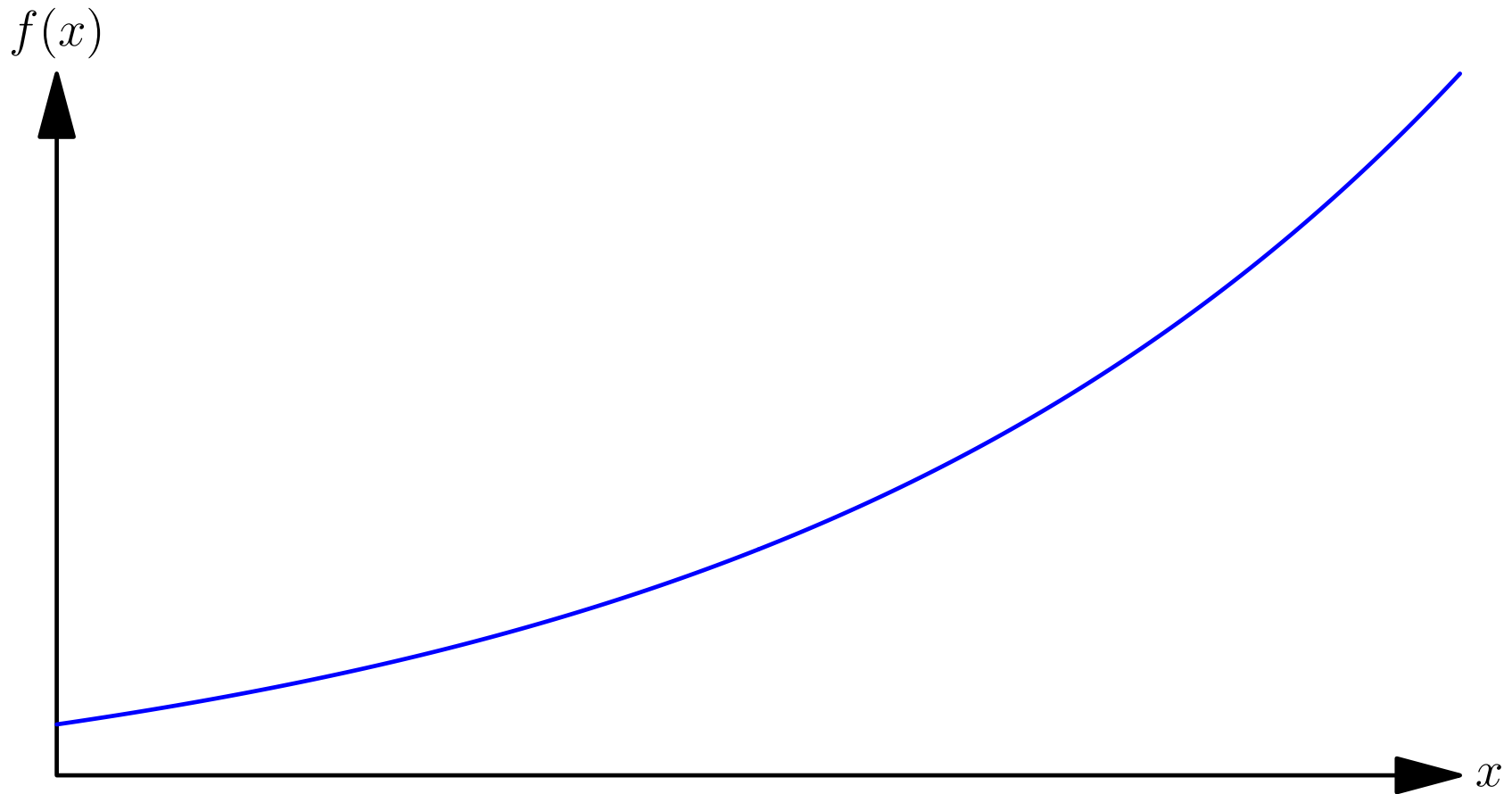
Lipschitz Function

- One way to characterise a well-behaved function, $f(x)$, is if there exists a number $K < \infty$ such that for all x and y

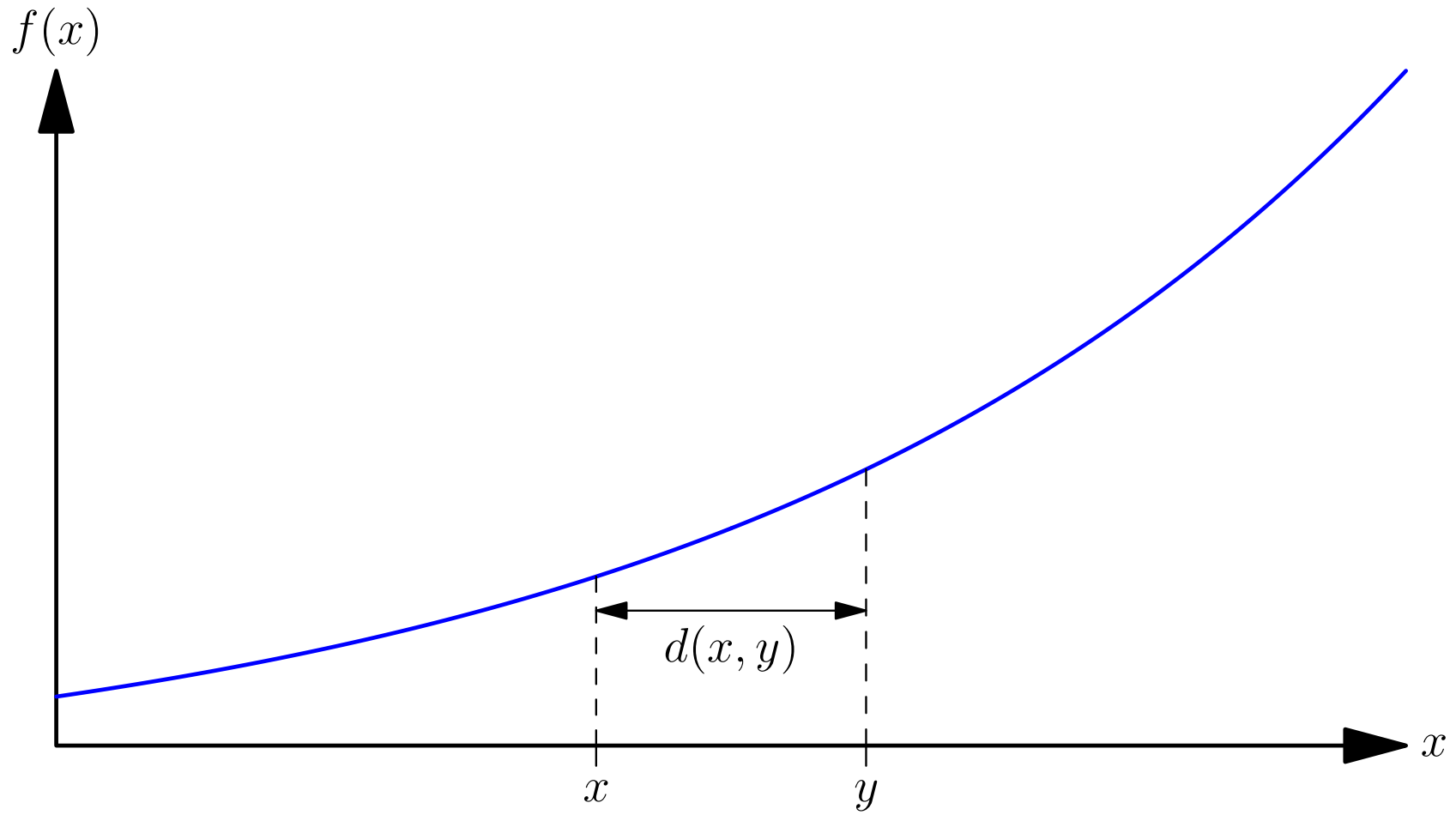
$$d(f(x), f(y)) \leq K d(x, y)$$

- This is known as a **Lipschitz condition** and the function is said to be K -Lipschitz or Lipschitz continuous
- Note that such functions cannot have any jumps (i.e. they are continuous)
- The size of K measures the limit on the amplifying effect of the function

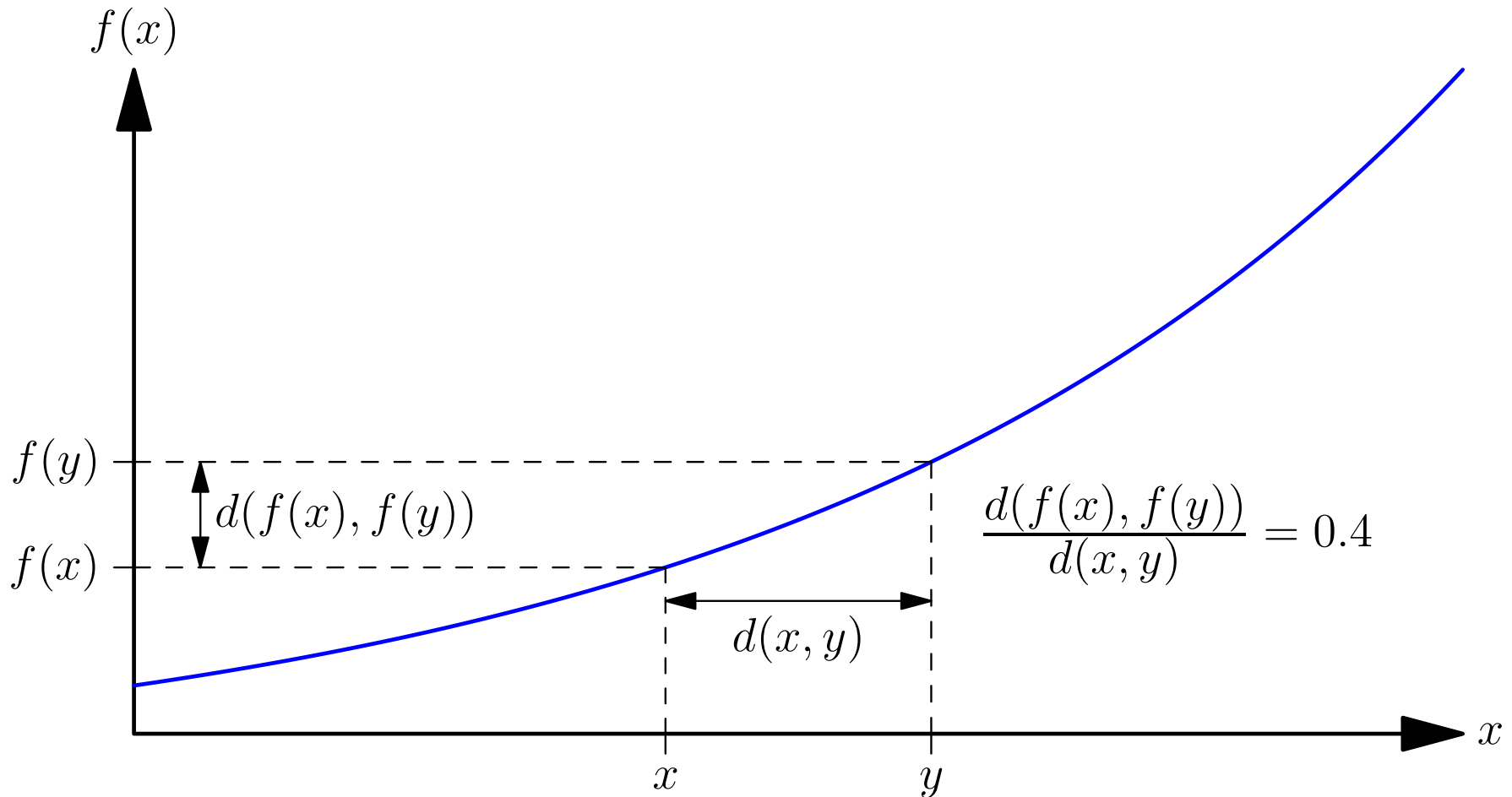
Lipschitz Continuous Functions



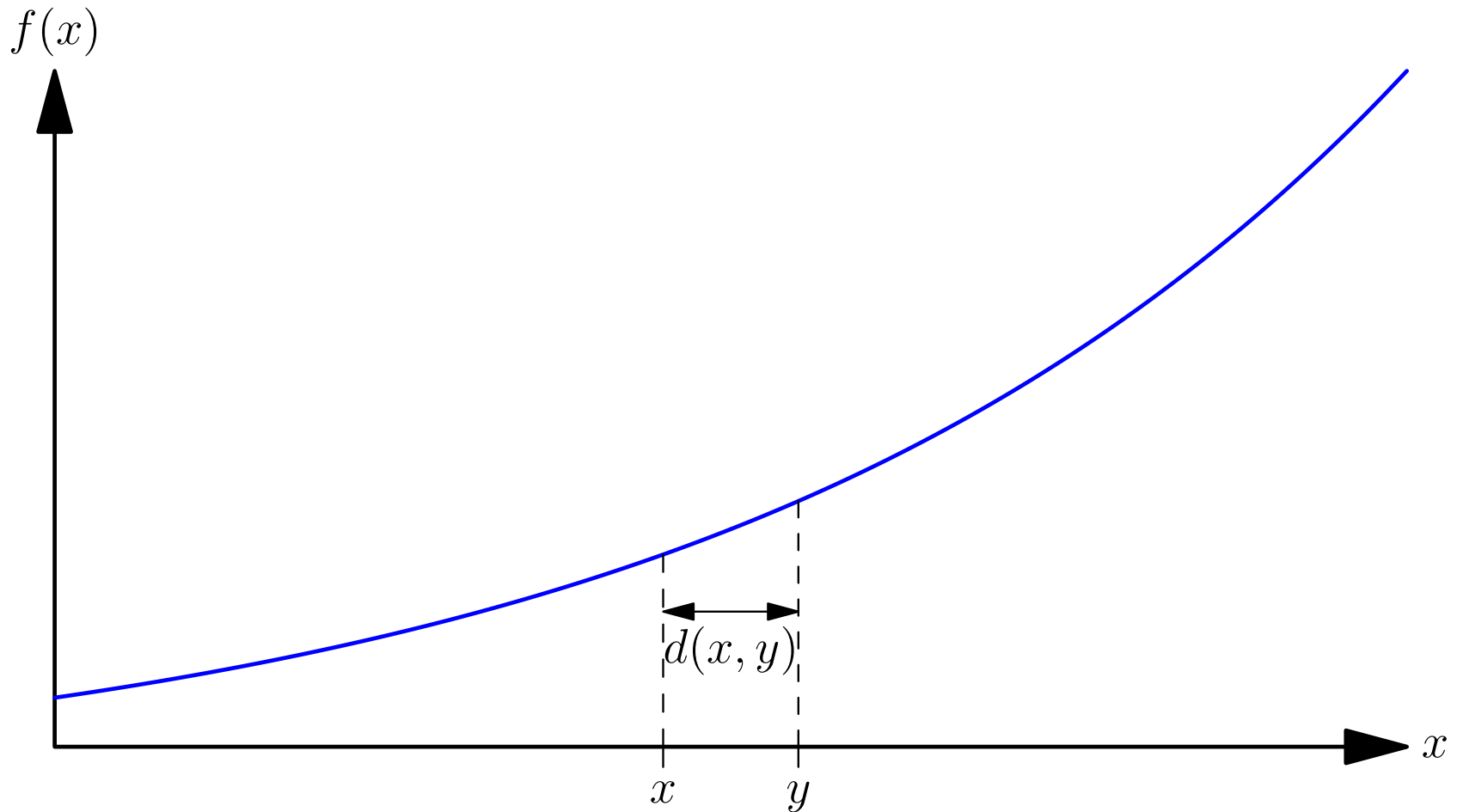
Lipschitz Continuous Functions



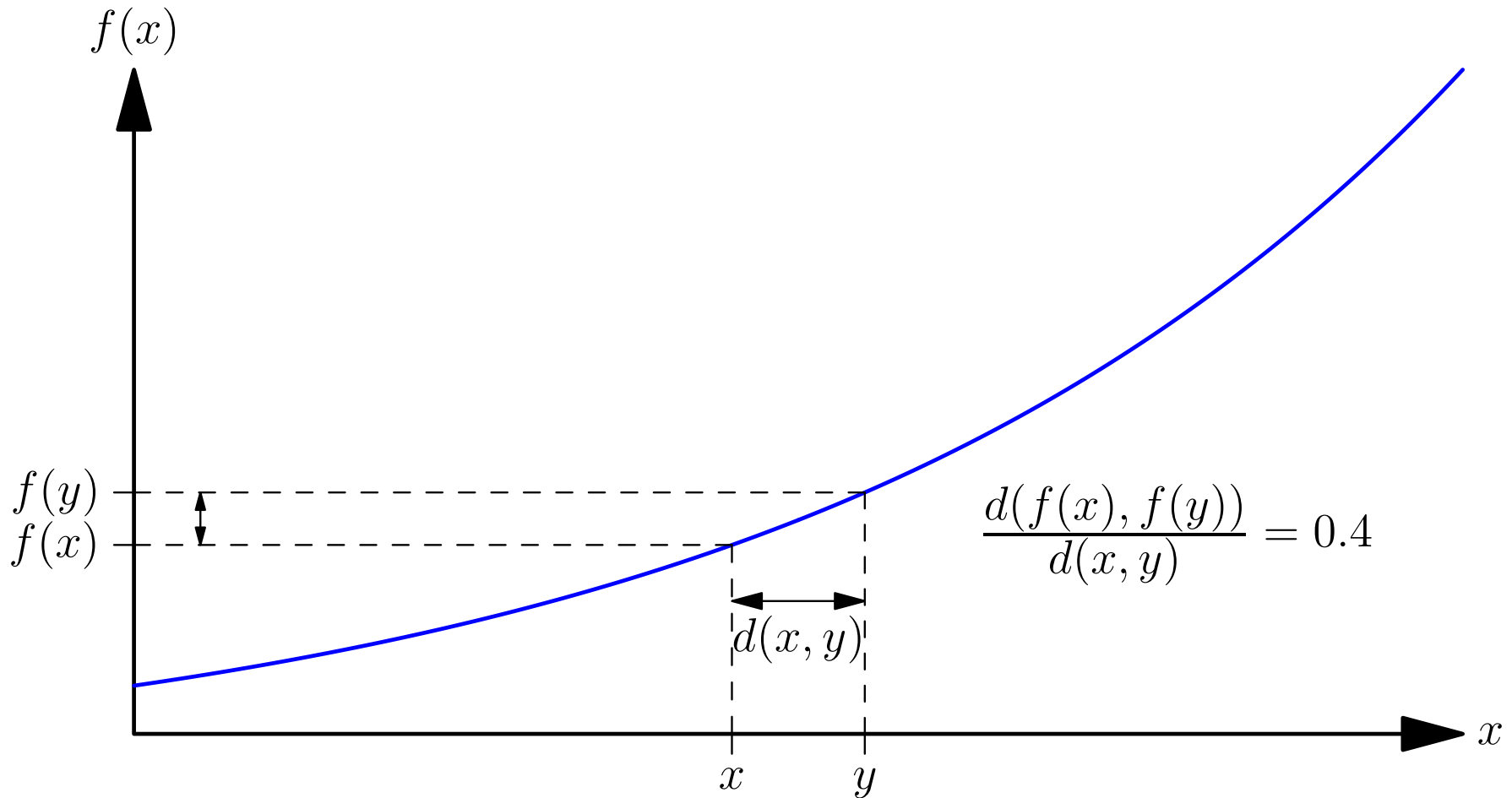
Lipschitz Continuous Functions



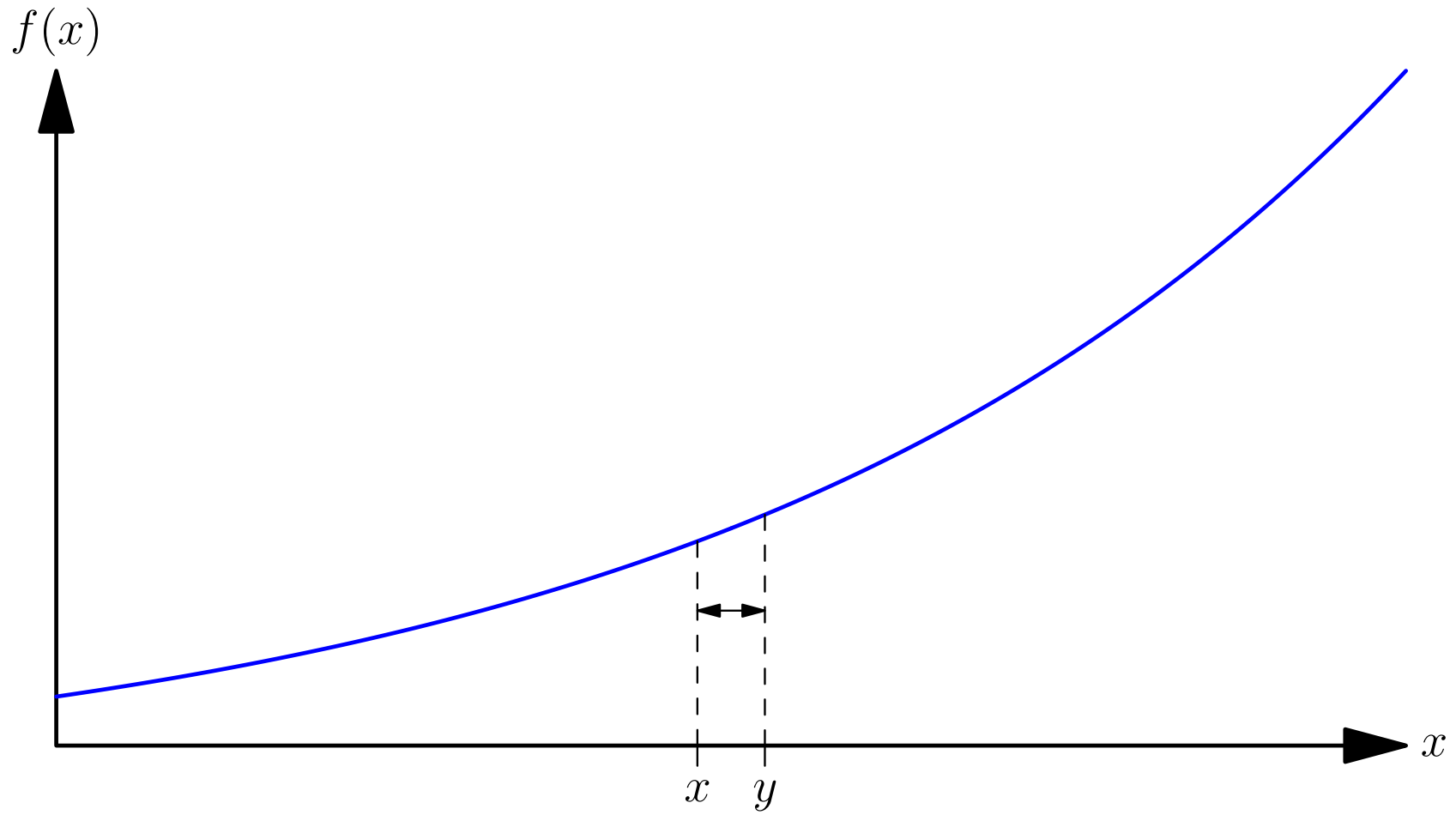
Lipschitz Continuous Functions



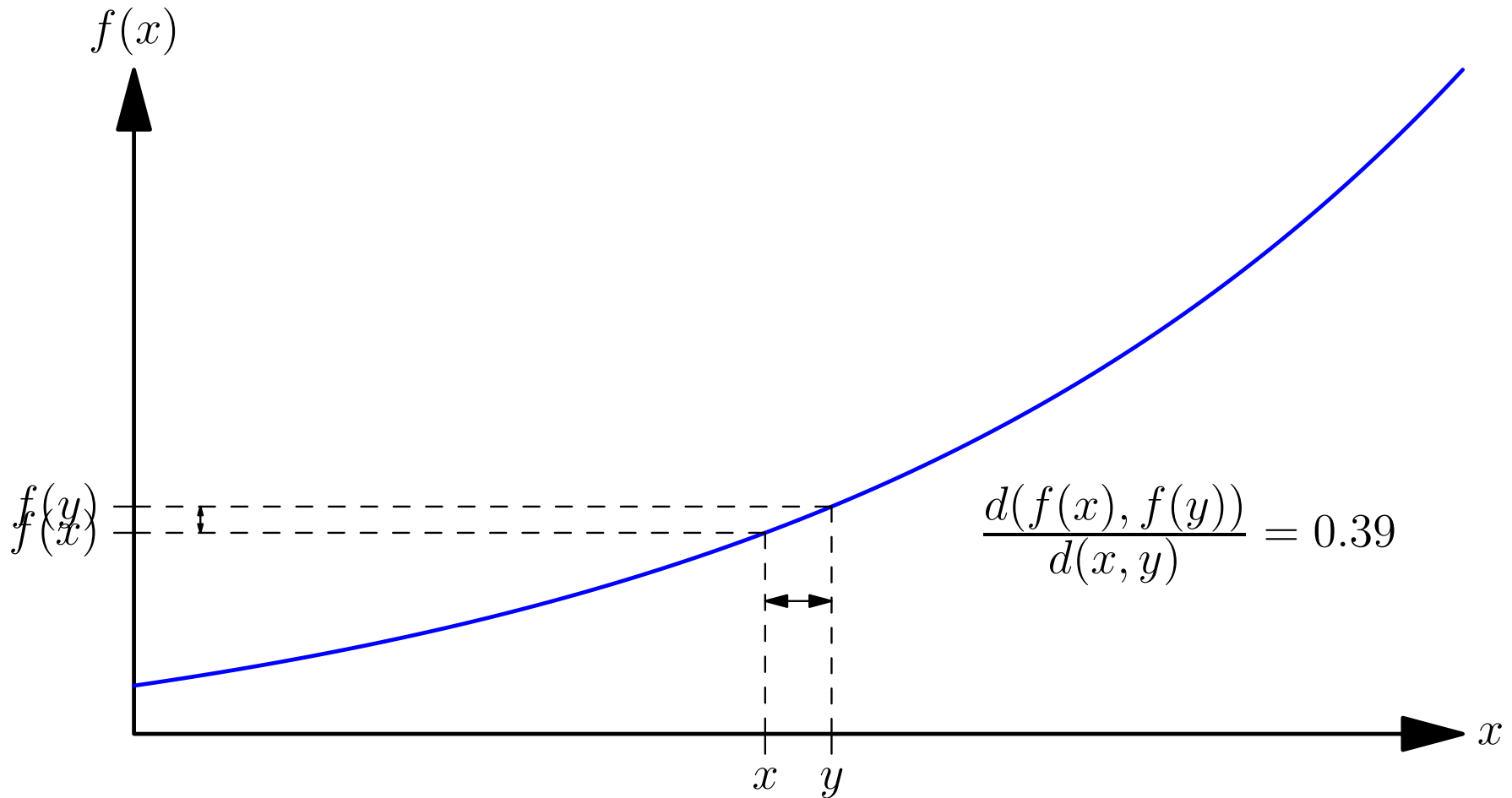
Lipschitz Continuous Functions



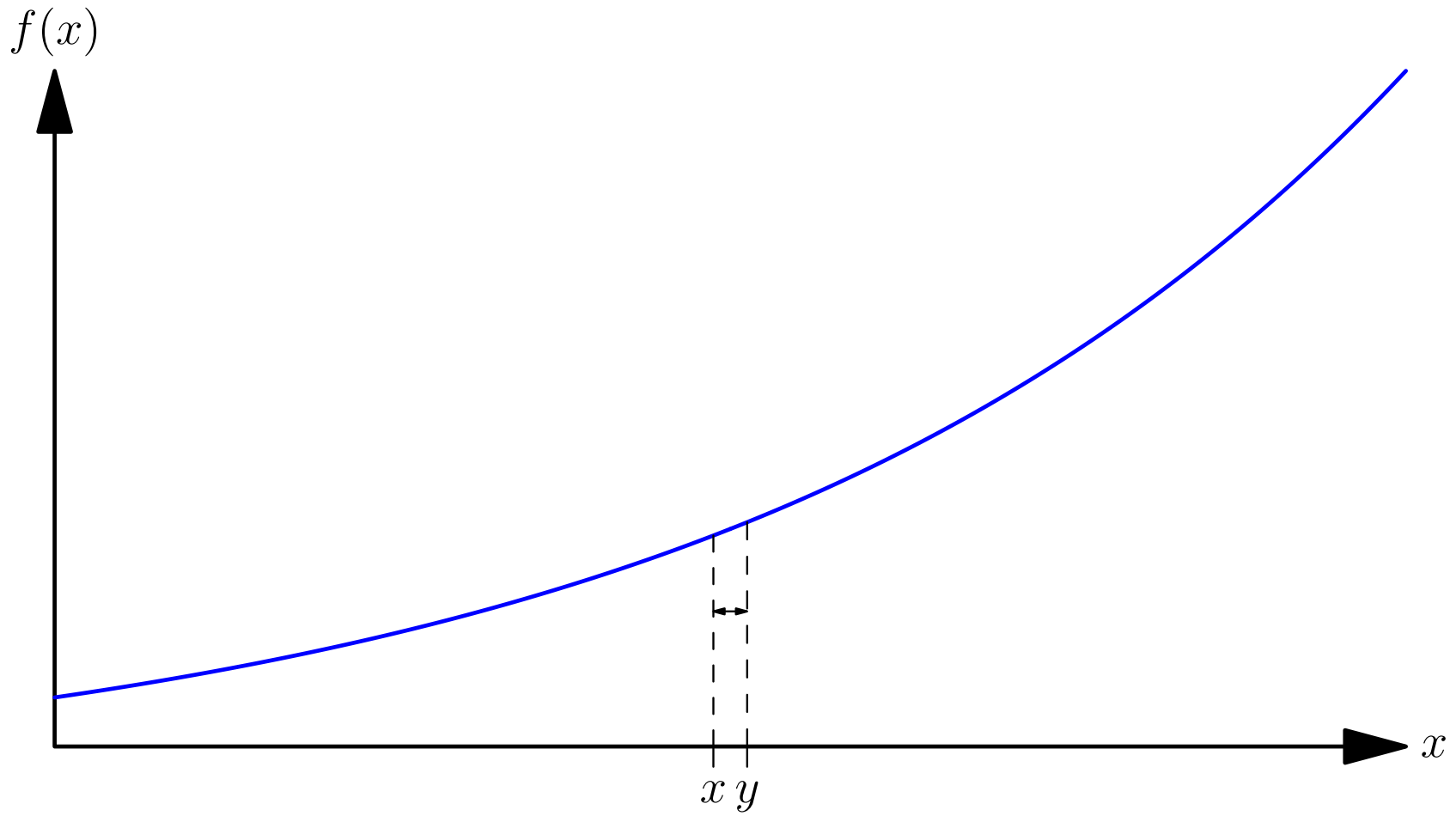
Lipschitz Continuous Functions



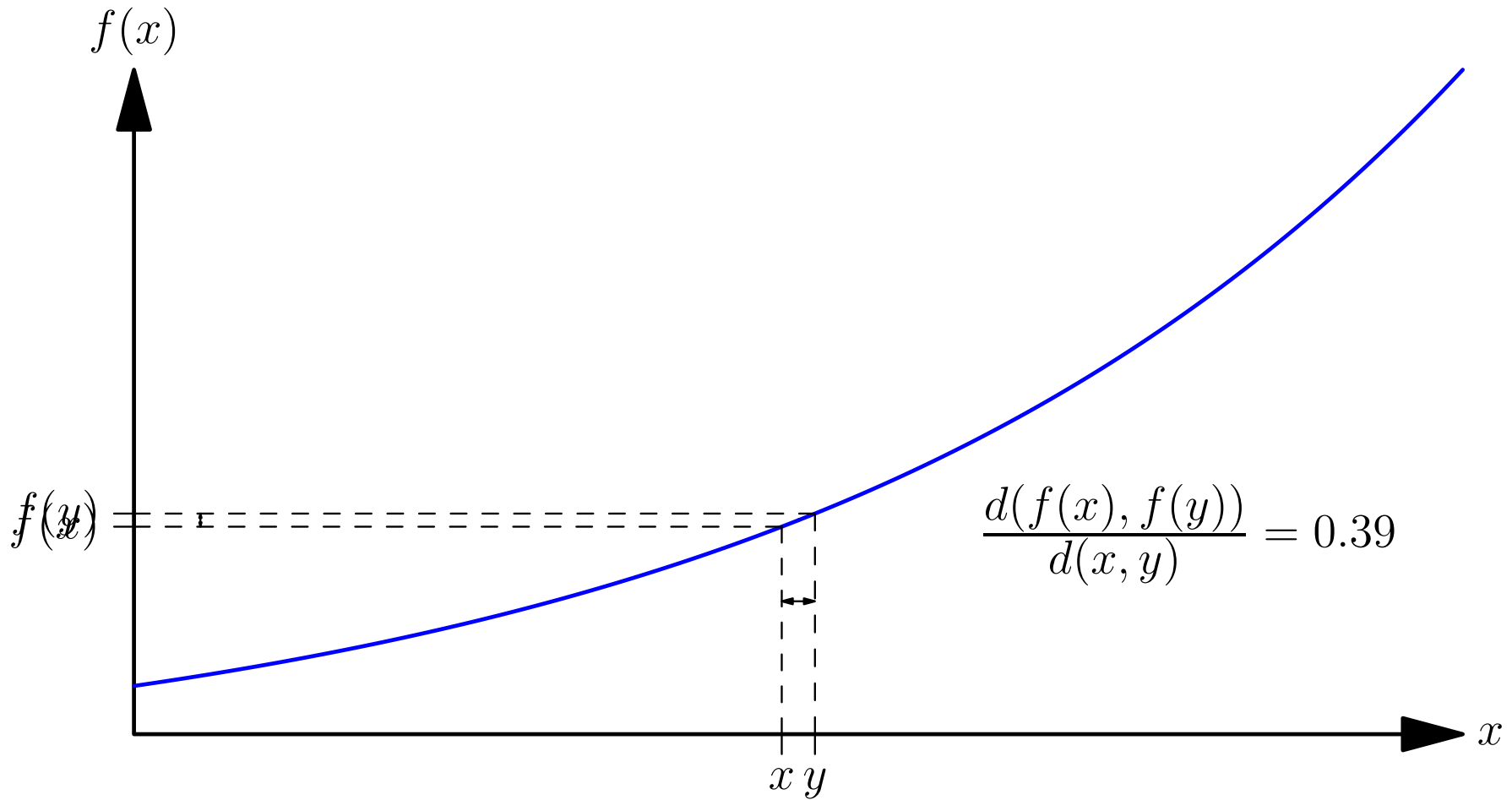
Lipschitz Continuous Functions



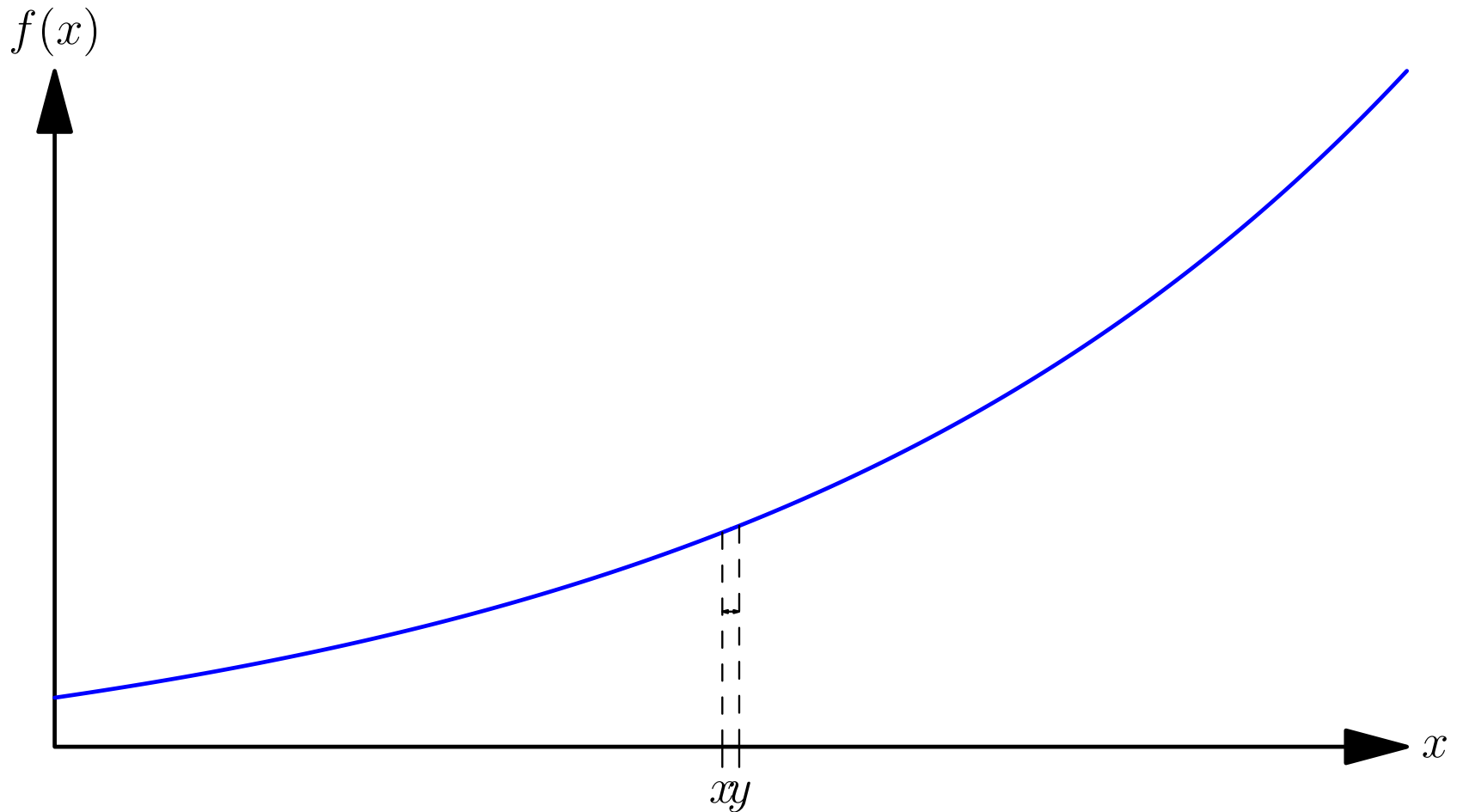
Lipschitz Continuous Functions



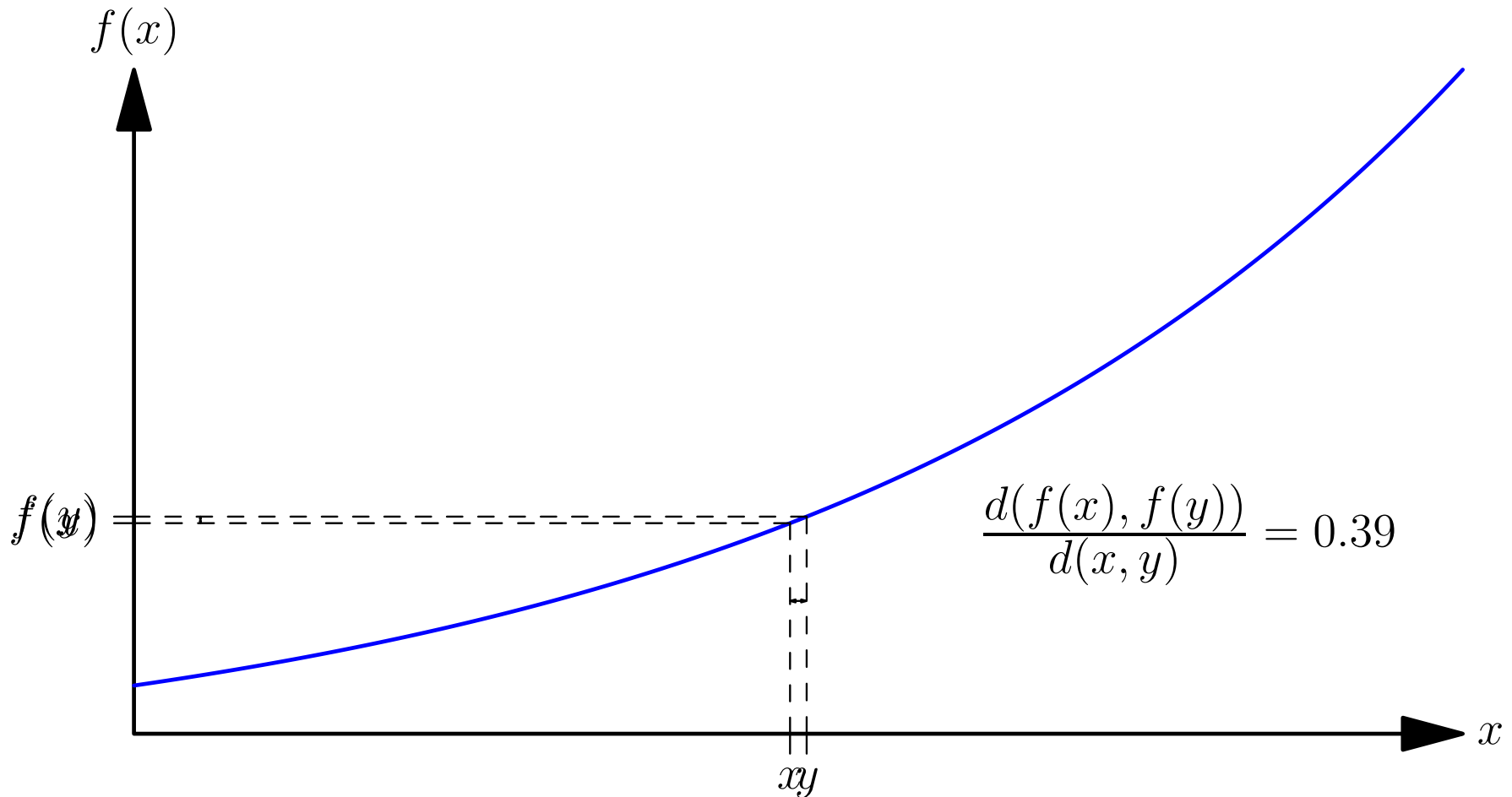
Lipschitz Continuous Functions



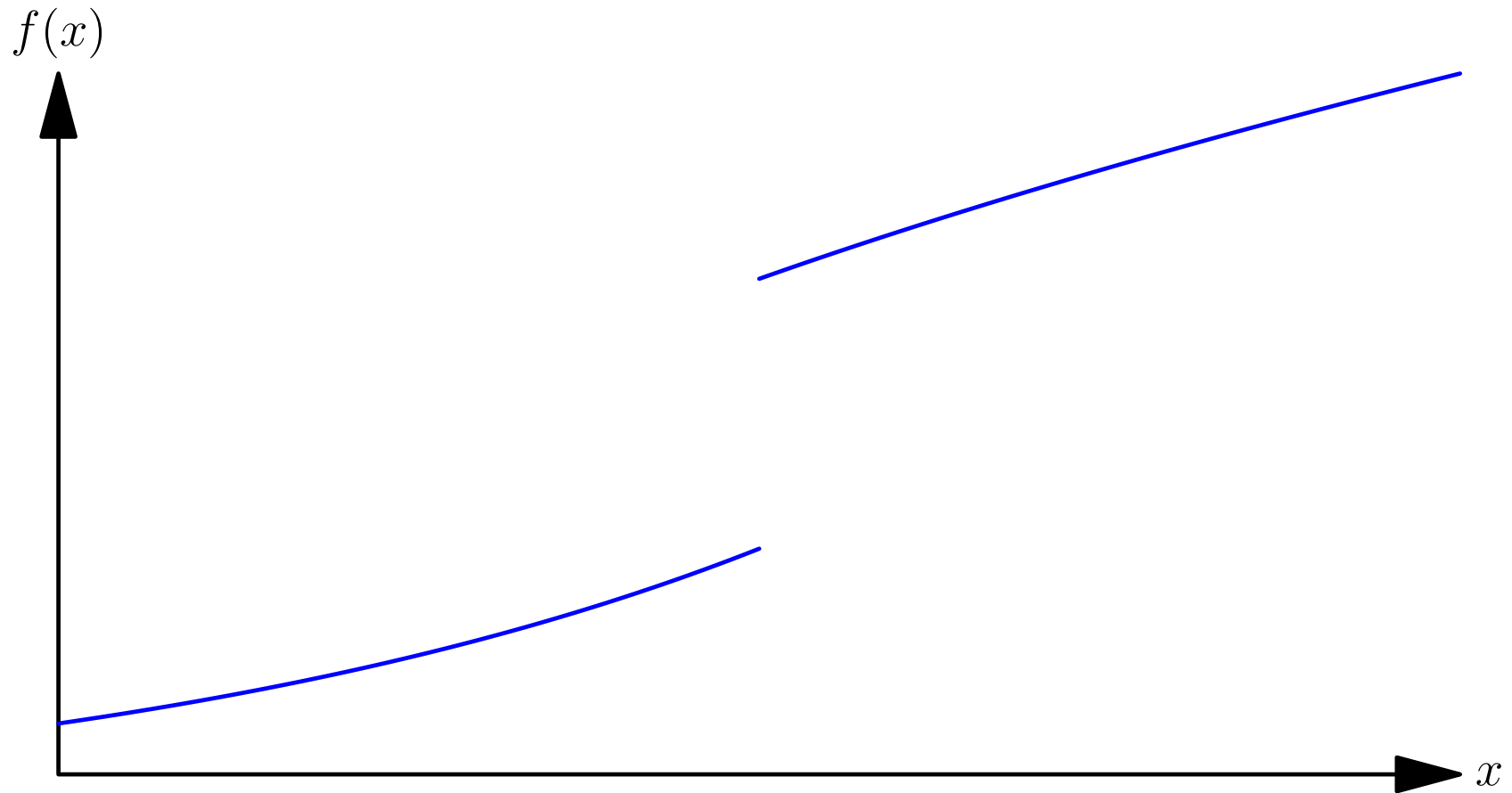
Lipschitz Continuous Functions



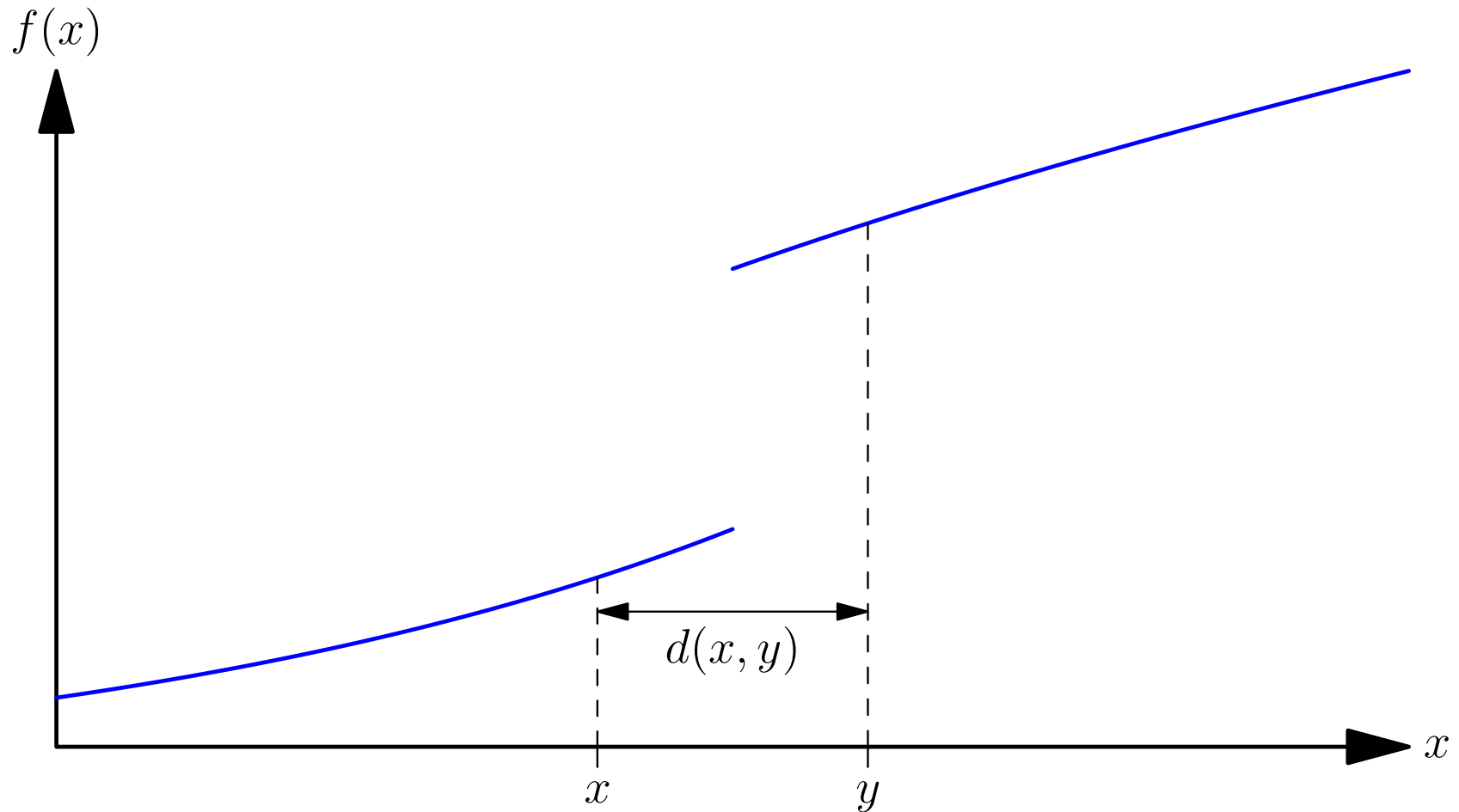
Lipschitz Continuous Functions



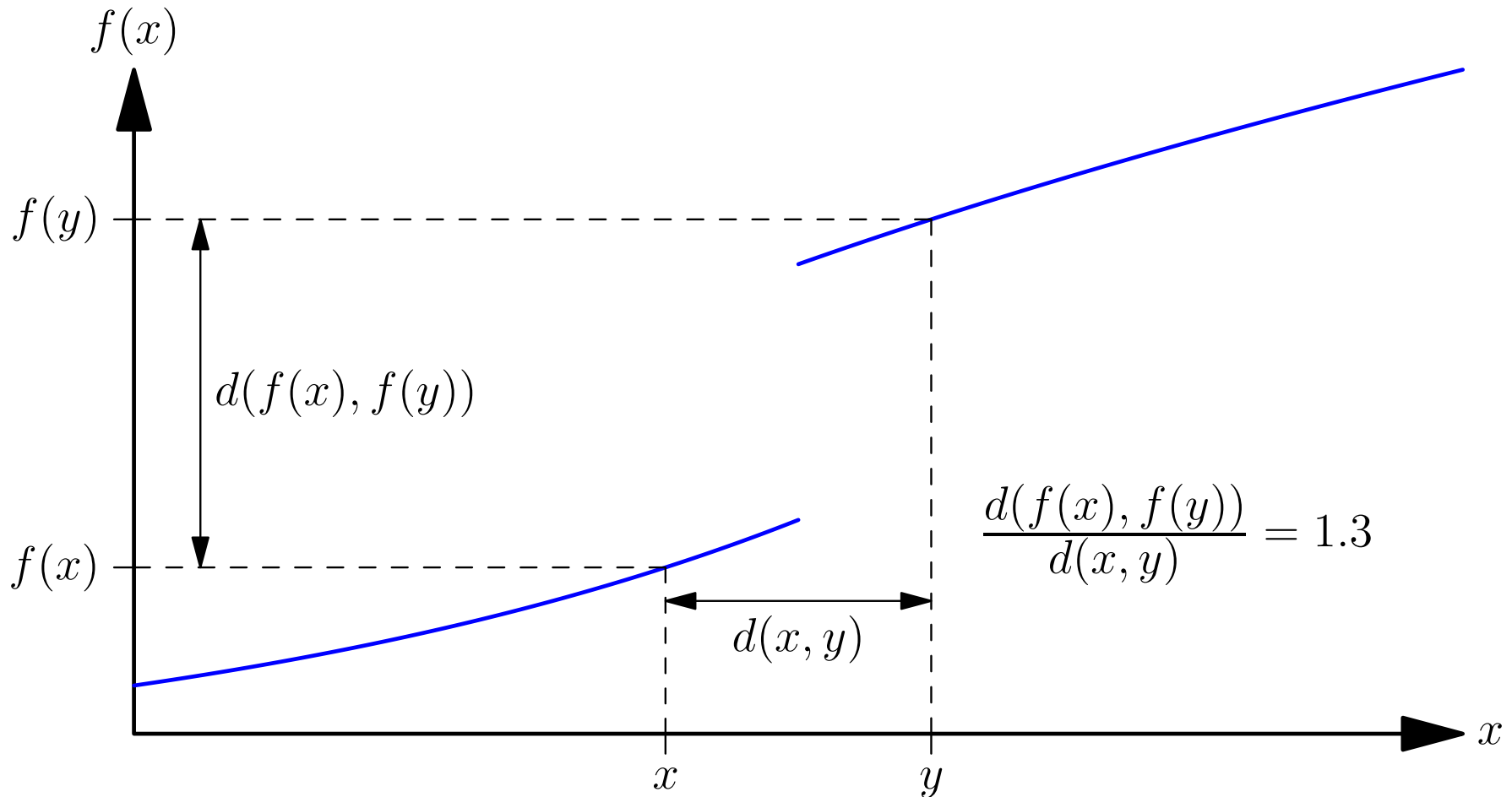
Lipschitz Continuous Functions



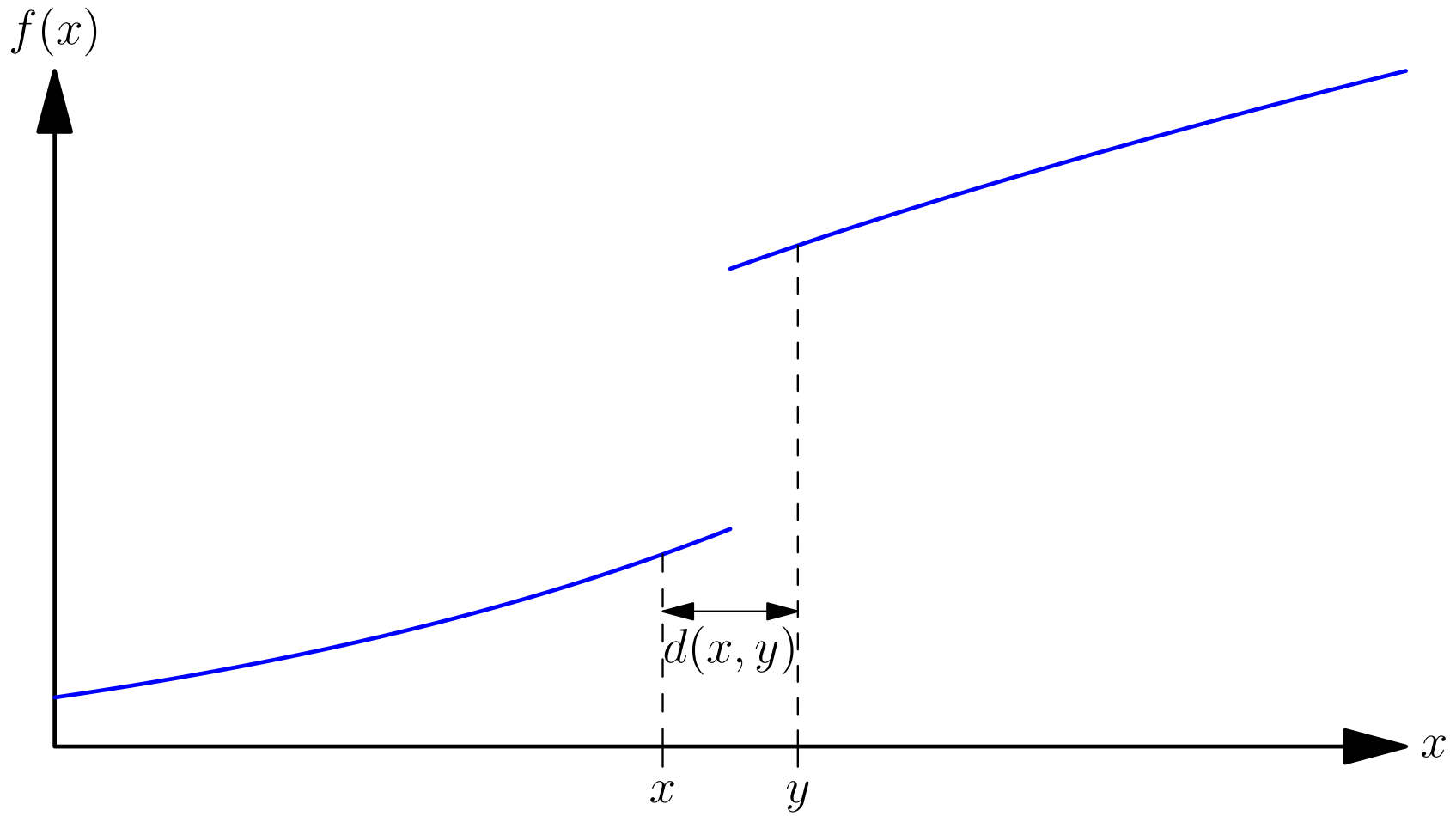
Lipschitz Continuous Functions



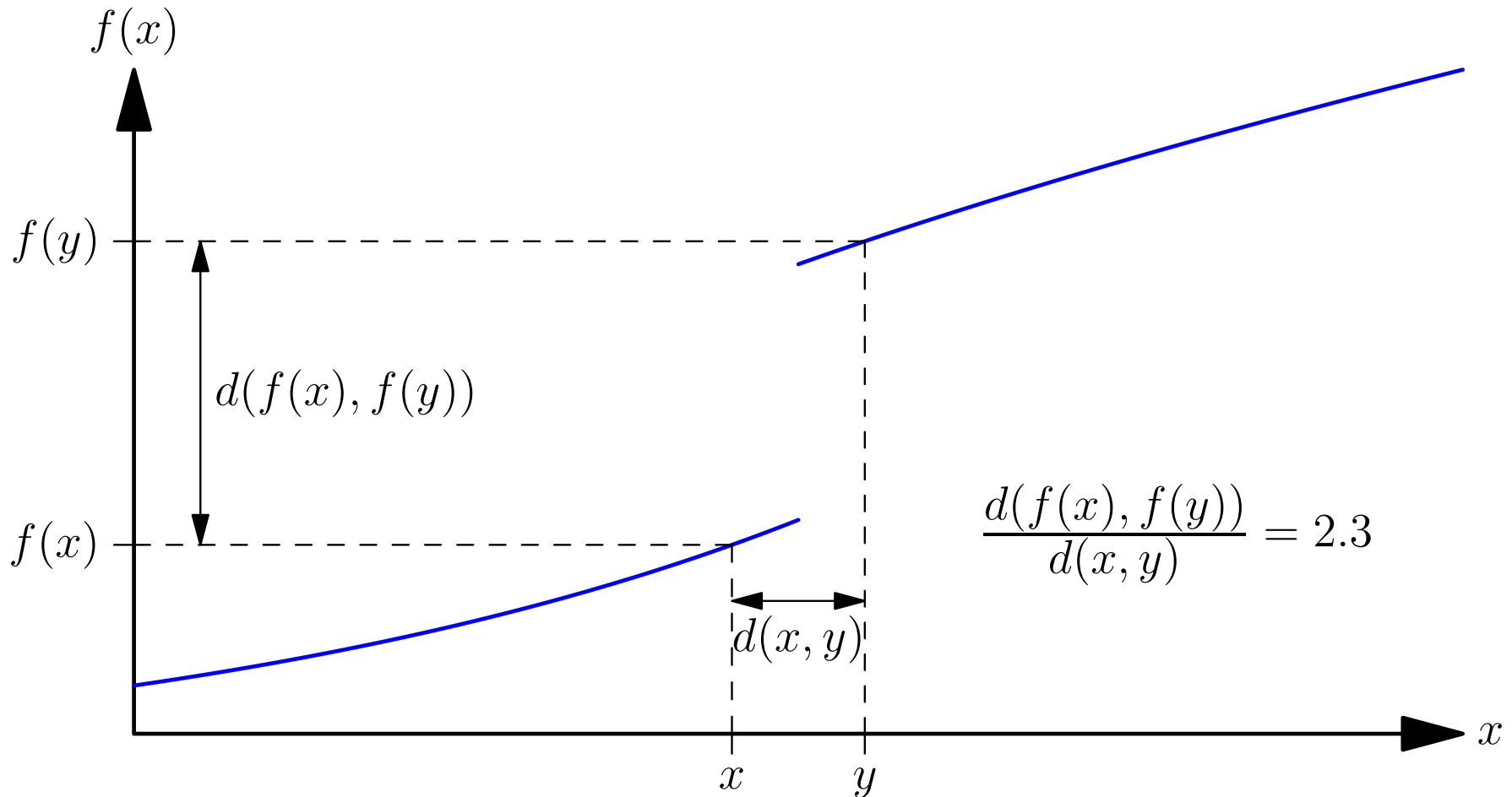
Lipschitz Continuous Functions



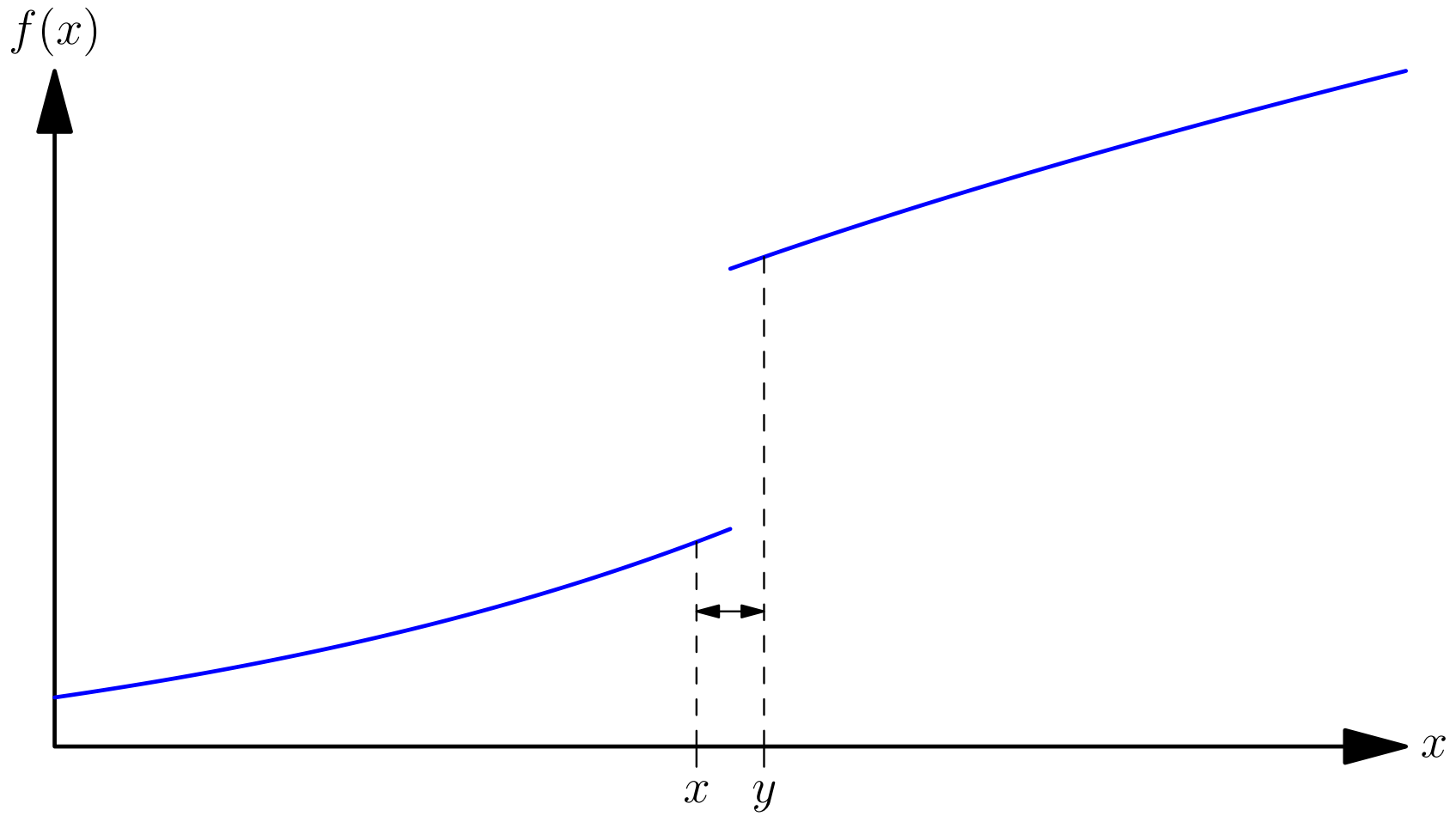
Lipschitz Continuous Functions



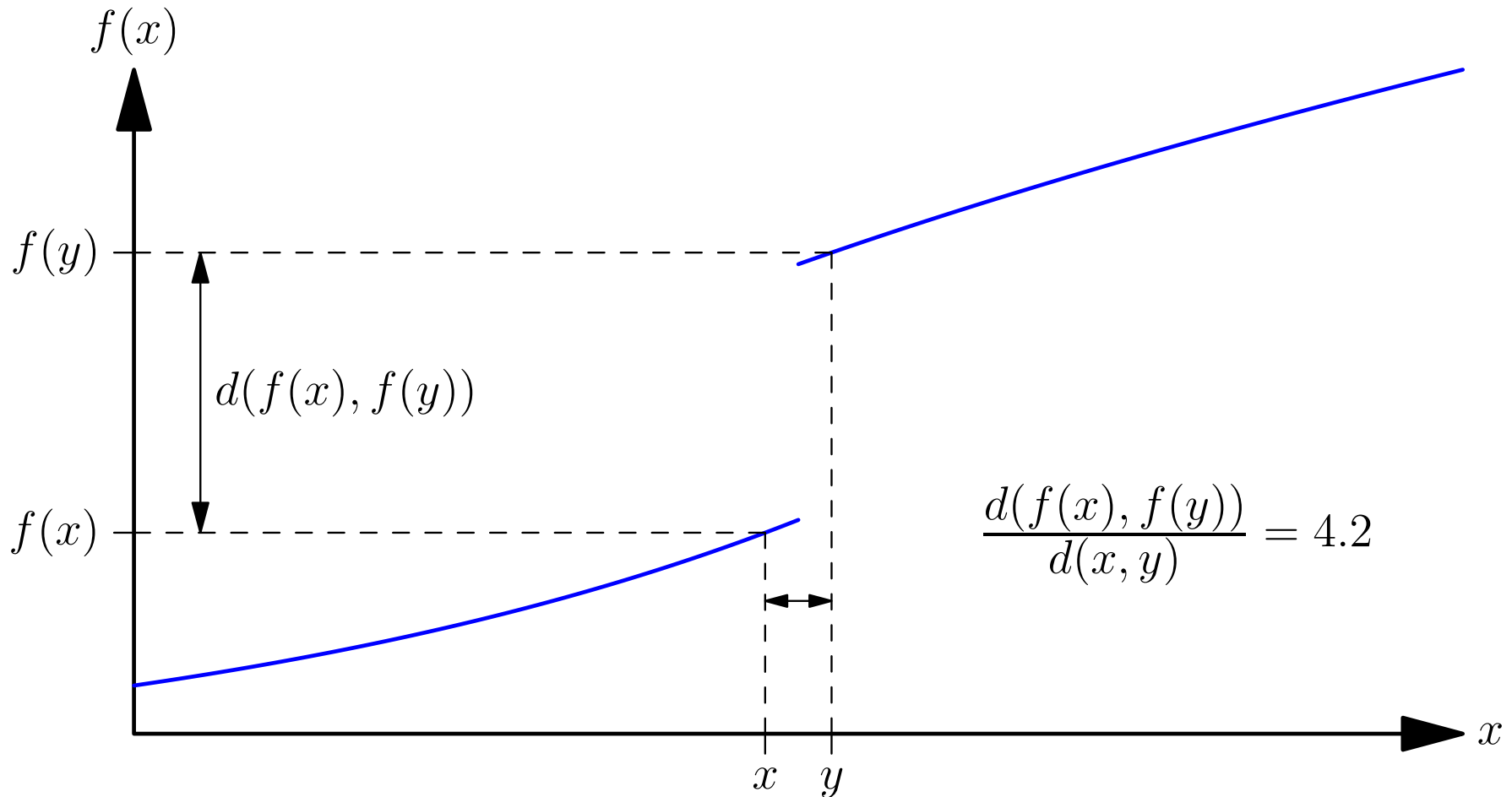
Lipschitz Continuous Functions



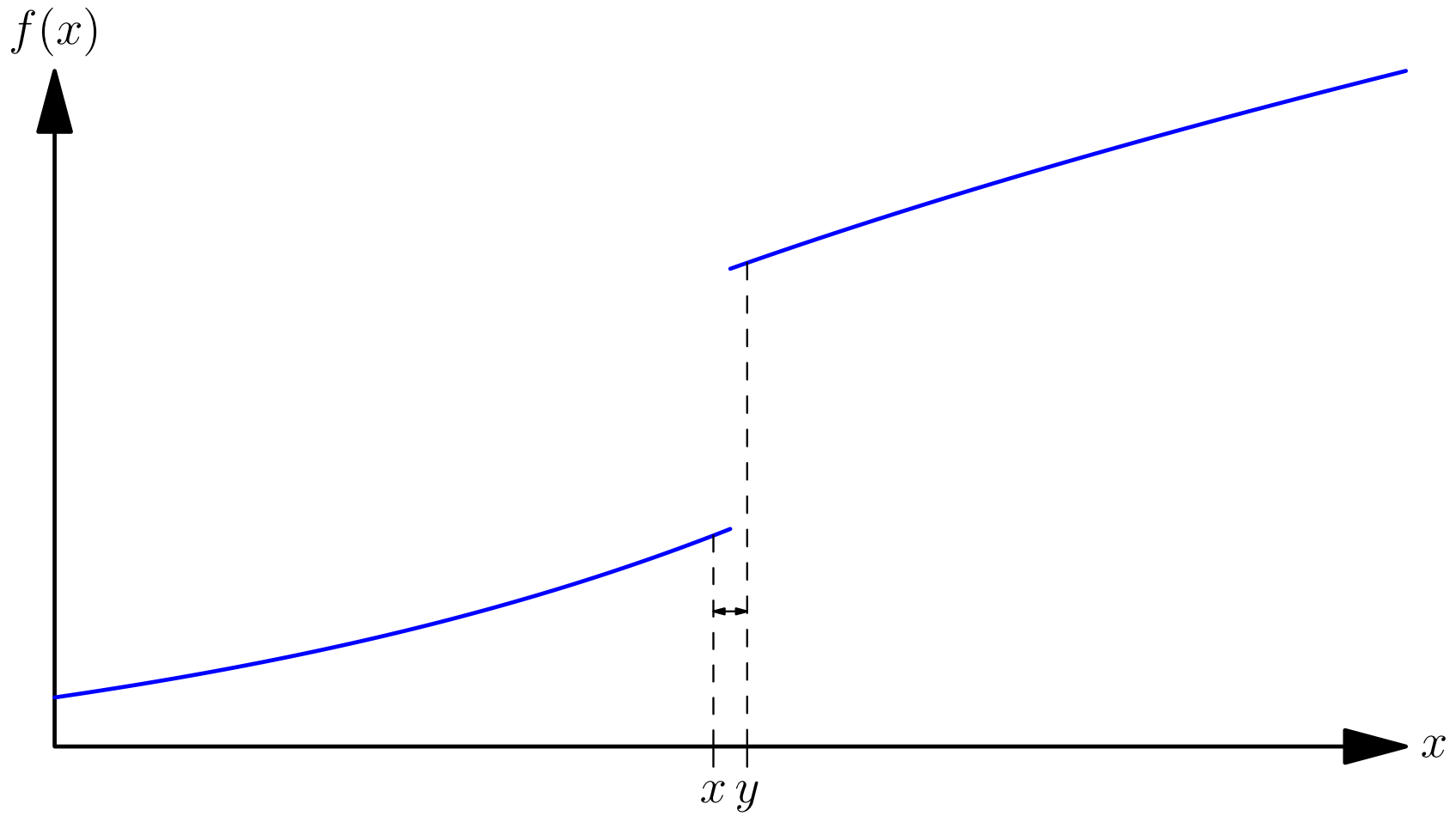
Lipschitz Continuous Functions



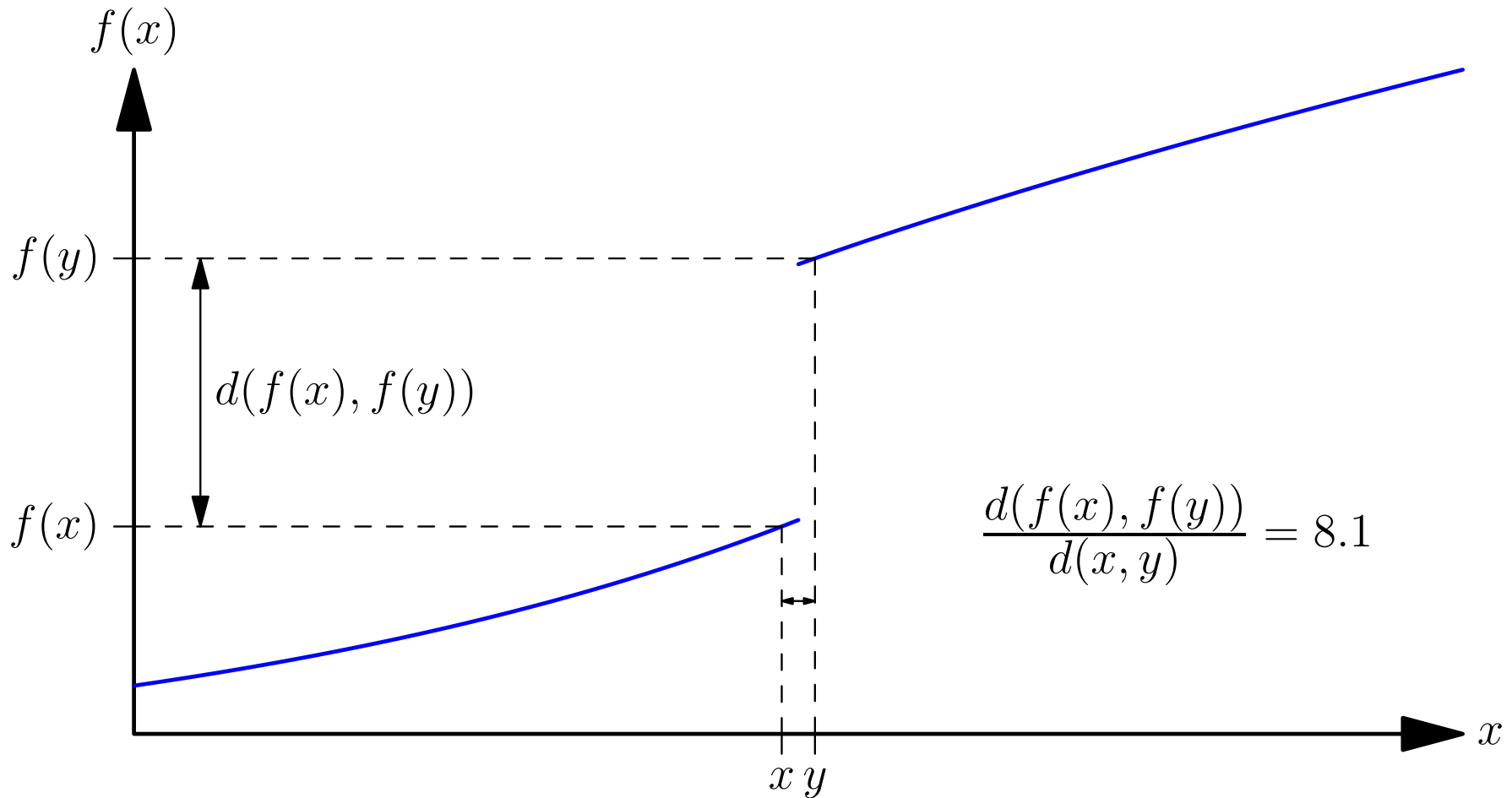
Lipschitz Continuous Functions



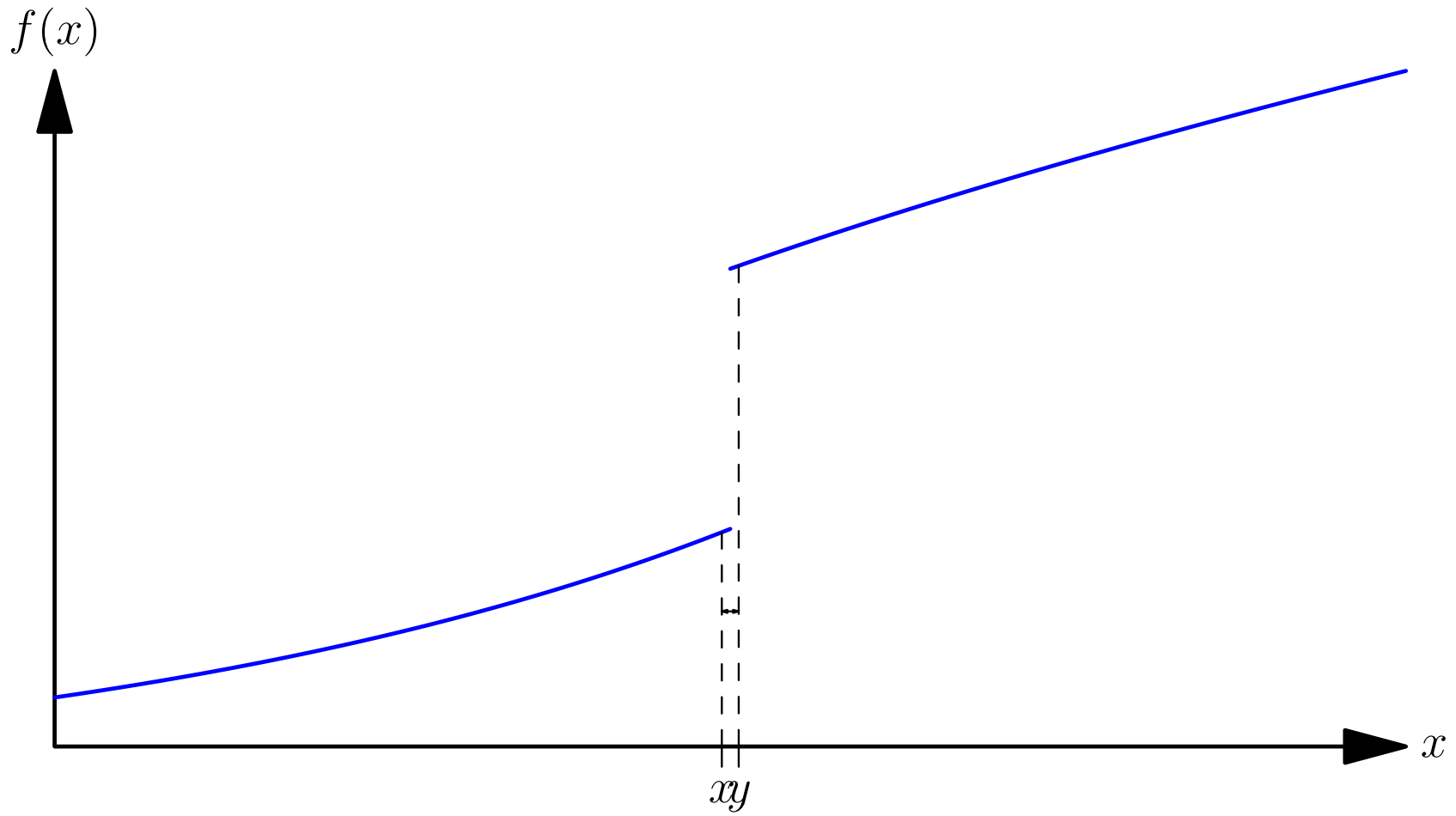
Lipschitz Continuous Functions



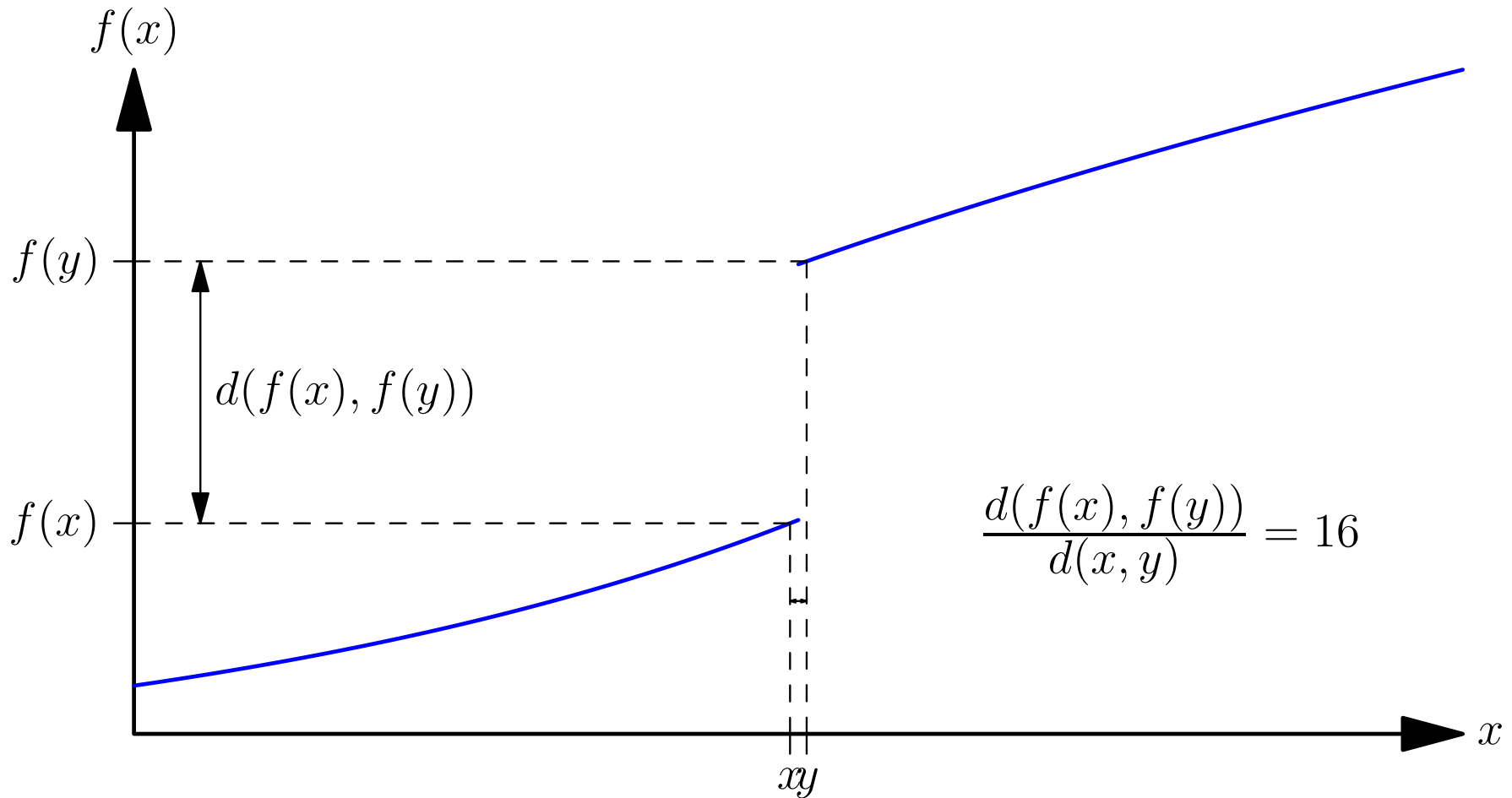
Lipschitz Continuous Functions



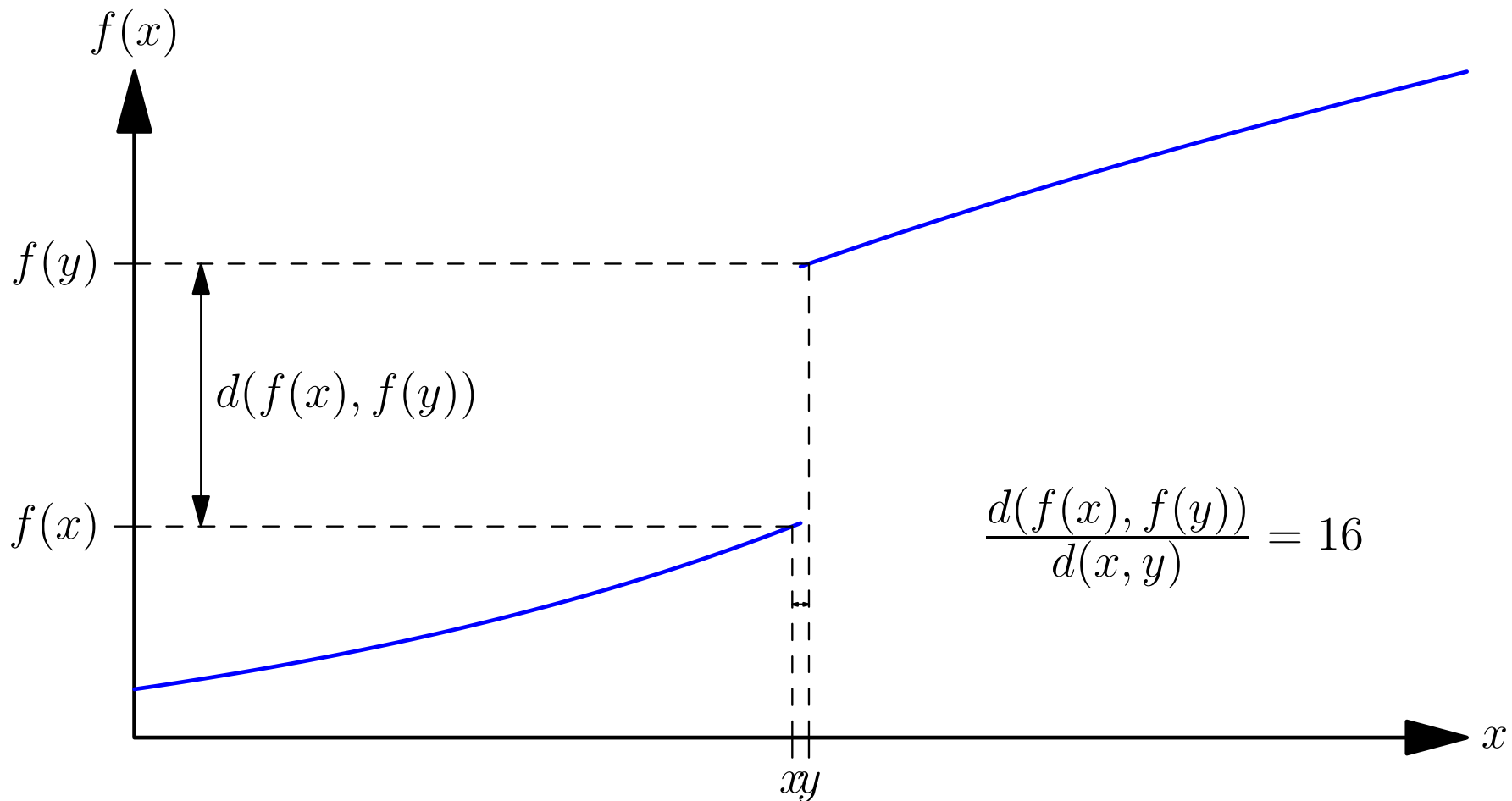
Lipschitz Continuous Functions



Lipschitz Continuous Functions



Lipschitz Continuous Functions



- The definition applies to any function defined in a metric space

Contractive Mappings

- An interesting class of functions are those for which $K < 1$
- These are said to be contractive mappings
- A famous theorem that applies to contractive mappings is the Banach fixed-point theorem which says there exists a unique fixed point such that $f(x) = x$, and repeatedly applying f converges to it
- This is used for example in showing that various algorithms will converge

Contractive Mappings

- An interesting class of functions are those for which $K < 1$
- These are said to be contractive mappings
- A famous theorem that applies to contractive mappings is the Banach fixed-point theorem which says there exists a unique fixed point such that $f(x) = x$, and repeatedly applying f converges to it
- This is used for example in showing that various algorithms will converge

Contractive Mappings

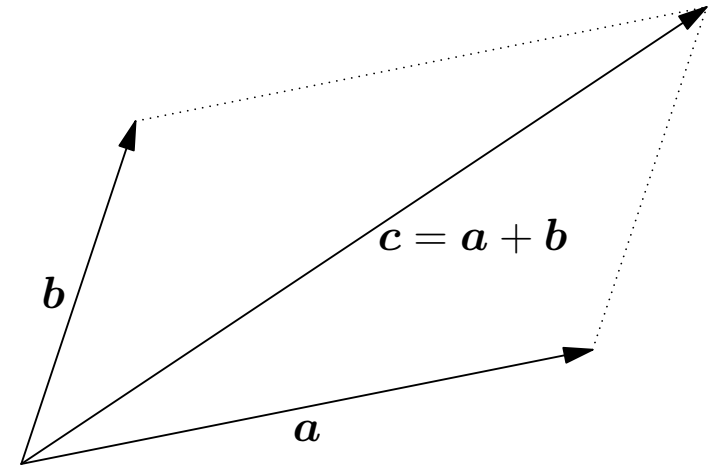
- An interesting class of functions are those for which $K < 1$
- These are said to be contractive mappings
- A famous theorem that applies to contractive mappings is the Banach fixed-point theorem which says there exists a unique fixed point such that $f(x) = x$, and repeatedly applying f converges to it
- This is used for example in showing that various algorithms will converge

Contractive Mappings

- An interesting class of functions are those for which $K < 1$
- These are said to be contractive mappings
- A famous theorem that applies to contractive mappings is the Banach fixed-point theorem which says there exists a unique fixed point such that $f(x) = x$, and repeatedly applying f converges to it
- This is used for example in showing that various algorithms will converge

Outline

1. Vector Spaces
2. Metrics (distances)
3. **Norms**



Norms

- Vector spaces are even more interesting with a notion of length
- **Norms** provide some measure of the size of a vector
- To formalise this we define the **norm** of an object $\mathbf{v}$ as $\|\mathbf{v}\|$ satisfying
 1. $\|\mathbf{v}\| > 0$ if $\mathbf{v} \neq \mathbf{0}$ and $\|\mathbf{0}\| = 0$ (non-negativity)
 2. $\|a\mathbf{v}\| = |a|\|\mathbf{v}\|$ (“linearity”)
 3. $\|\mathbf{u} + \mathbf{v}\| \leq \|\mathbf{u}\| + \|\mathbf{v}\|$ (triangular inequality)
- When some criteria aren’t satisfied we have a **pseudo-norm**
- Norms provide a metric $d(\mathbf{x}, \mathbf{y}) = \|\mathbf{x} - \mathbf{y}\|$ (they are metric spaces)

Norms

- Vector spaces are even more interesting with a notion of length
- **Norms** provide some measure of the size of a vector
- To formalise this we define the **norm** of an object $\mathbf{v}$ as $\|\mathbf{v}\|$ satisfying
 1. $\|\mathbf{v}\| > 0$ if $\mathbf{v} \neq \mathbf{0}$ and $\|\mathbf{0}\| = 0$ (non-negativity)
 2. $\|a\mathbf{v}\| = |a|\|\mathbf{v}\|$ (“linearity”)
 3. $\|\mathbf{u} + \mathbf{v}\| \leq \|\mathbf{u}\| + \|\mathbf{v}\|$ (triangular inequality)
- When some criteria aren’t satisfied we have a **pseudo-norm**
- Norms provide a metric $d(\mathbf{x}, \mathbf{y}) = \|\mathbf{x} - \mathbf{y}\|$ (they are metric spaces)

Norms

- Vector spaces are even more interesting with a notion of length
- **Norms** provide some measure of the size of a vector
- To formalise this we define the **norm** of an object $\mathbf{v}$ as $\|\mathbf{v}\|$ satisfying
 1. $\|\mathbf{v}\| > 0$ if $\mathbf{v} \neq \mathbf{0}$ and $\|\mathbf{0}\| = 0$ (non-negativity)
 2. $\|a\mathbf{v}\| = |a|\|\mathbf{v}\|$ (“linearity”)
 3. $\|\mathbf{u} + \mathbf{v}\| \leq \|\mathbf{u}\| + \|\mathbf{v}\|$ (triangular inequality)
- When some criteria aren’t satisfied we have a **pseudo-norm**
- Norms provide a metric $d(\mathbf{x}, \mathbf{y}) = \|\mathbf{x} - \mathbf{y}\|$ (they are metric spaces)

Norms

- Vector spaces are even more interesting with a notion of length
- **Norms** provide some measure of the size of a vector
- To formalise this we define the **norm** of an object $\mathbf{v}$ as $\|\mathbf{v}\|$ satisfying
 1. $\|\mathbf{v}\| > 0$ if $\mathbf{v} \neq \mathbf{0}$ and $\|\mathbf{0}\| = 0$ (non-negativity)
 2. $\|a\mathbf{v}\| = |a|\|\mathbf{v}\|$ (“linearity”)
 3. $\|\mathbf{u} + \mathbf{v}\| \leq \|\mathbf{u}\| + \|\mathbf{v}\|$ (triangular inequality)
- When some criteria aren’t satisfied we have a **pseudo-norm**
- Norms provide a metric $d(\mathbf{x}, \mathbf{y}) = \|\mathbf{x} - \mathbf{y}\|$ (they are metric spaces)

Norms

- Vector spaces are even more interesting with a notion of length
- **Norms** provide some measure of the size of a vector
- To formalise this we define the **norm** of an object $\mathbf{v}$ as $\|\mathbf{v}\|$ satisfying
 1. $\|\mathbf{v}\| > 0$ if $\mathbf{v} \neq \mathbf{0}$ and $\|\mathbf{0}\| = 0$ (non-negativity)
 2. $\|a\mathbf{v}\| = |a|\|\mathbf{v}\|$ (“linearity”)
 3. $\|\mathbf{u} + \mathbf{v}\| \leq \|\mathbf{u}\| + \|\mathbf{v}\|$ (triangular inequality)
- When some criteria aren’t satisfied we have a **pseudo-norm**
- Norms provide a metric $d(\mathbf{x}, \mathbf{y}) = \|\mathbf{x} - \mathbf{y}\|$ (they are metric spaces)

Norms

- Vector spaces are even more interesting with a notion of length
- **Norms** provide some measure of the size of a vector
- To formalise this we define the **norm** of an object $\mathbf{v}$ as $\|\mathbf{v}\|$ satisfying
 1. $\|\mathbf{v}\| > 0$ if $\mathbf{v} \neq \mathbf{0}$ and $\|\mathbf{0}\| = 0$ (non-negativity)
 2. $\|a\mathbf{v}\| = |a|\|\mathbf{v}\|$ (“linearity”)
 3. $\|\mathbf{u} + \mathbf{v}\| \leq \|\mathbf{u}\| + \|\mathbf{v}\|$ (triangular inequality)
- When some criteria aren't satisfied we have a **pseudo-norm**
- Norms provide a metric $d(\mathbf{x}, \mathbf{y}) = \|\mathbf{x} - \mathbf{y}\|$ (they are metric spaces)

Norms

- Vector spaces are even more interesting with a notion of length
- **Norms** provide some measure of the size of a vector
- To formalise this we define the **norm** of an object $\mathbf{v}$ as $\|\mathbf{v}\|$ satisfying
 1. $\|\mathbf{v}\| > 0$ if $\mathbf{v} \neq \mathbf{0}$ and $\|\mathbf{0}\| = 0$ (non-negativity)
 2. $\|a\mathbf{v}\| = |a|\|\mathbf{v}\|$ (“linearity”)
 3. $\|\mathbf{u} + \mathbf{v}\| \leq \|\mathbf{u}\| + \|\mathbf{v}\|$ (triangular inequality)
- When some criteria aren’t satisfied we have a **pseudo-norm**
- Norms provide a metric $d(\mathbf{x}, \mathbf{y}) = \|\mathbf{x} - \mathbf{y}\|$ (they are metric spaces)

Vector Norms

- The familiar vector norm is the (Euclidean) two norm

$$\|\mathbf{v}\|_2 = \sqrt{v_1^2 + v_2^2 + \cdots + v_n^2}$$

- Other norms exist, such as the p -norm ($p \geq 1$)

$$\|\mathbf{v}\|_p = \left(\sum_{i=1}^n |v_i|^p \right)^{1/p}$$

- Special cases include the 1-norm and the infinite norm

$$\|\mathbf{v}\|_1 = \sum_{i=1}^n |v_i| \qquad \|\mathbf{v}\|_\infty = \max_i |v_i|$$

- The 0-norm is a pseudo-norm as it does not satisfy condition 2

$$\|\mathbf{v}\|_0 = \text{number of non-zero components}$$

Vector Norms

- The familiar vector norm is the (Euclidean) two norm

$$\|\mathbf{v}\|_2 = \sqrt{v_1^2 + v_2^2 + \cdots + v_n^2}$$

- Other norms exist, such as the p -norm ($p \geq 1$)

$$\|\mathbf{v}\|_p = \left(\sum_{i=1}^n |v_i|^p \right)^{1/p}$$

- Special cases include the 1-norm and the infinite norm

$$\|\mathbf{v}\|_1 = \sum_{i=1}^n |v_i| \qquad \|\mathbf{v}\|_\infty = \max_i |v_i|$$

- The 0-norm is a pseudo-norm as it does not satisfy condition 2

$$\|\mathbf{v}\|_0 = \text{number of non-zero components}$$

Vector Norms

- The familiar vector norm is the (Euclidean) two norm

$$\|\mathbf{v}\|_2 = \sqrt{v_1^2 + v_2^2 + \cdots + v_n^2}$$

- Other norms exist, such as the p -norm ($p \geq 1$)

$$\|\mathbf{v}\|_p = \left(\sum_{i=1}^n |v_i|^p \right)^{1/p}$$

- Special cases include the 1-norm and the infinite norm

$$\|\mathbf{v}\|_1 = \sum_{i=1}^n |v_i| \qquad \|\mathbf{v}\|_\infty = \max_i |v_i|$$

- The 0-norm is a pseudo-norm as it does not satisfy condition 2

$$\|\mathbf{v}\|_0 = \text{number of non-zero components}$$

Vector Norms

- The familiar vector norm is the (Euclidean) two norm

$$\|\mathbf{v}\|_2 = \sqrt{v_1^2 + v_2^2 + \cdots + v_n^2}$$

- Other norms exist, such as the p -norm ($p \geq 1$)

$$\|\mathbf{v}\|_p = \left(\sum_{i=1}^n |v_i|^p \right)^{1/p}$$

- Special cases include the 1-norm and the infinite norm

$$\|\mathbf{v}\|_1 = \sum_{i=1}^n |v_i| \qquad \|\mathbf{v}\|_\infty = \max_i |v_i|$$

- The 0-norm is a pseudo-norm as it does not satisfy condition 2

$$\|\mathbf{v}\|_0 = \text{number of non-zero components}$$

Vector Norms

- The familiar vector norm is the (Euclidean) two norm

$$\|\mathbf{v}\|_2 = \sqrt{v_1^2 + v_2^2 + \cdots + v_n^2}$$

- Other norms exist, such as the p -norm ($p \geq 1$)

$$\|\mathbf{v}\|_p = \left(\sum_{i=1}^n |v_i|^p \right)^{1/p}$$

- Special cases include the 1-norm and the infinite norm

$$\|\mathbf{v}\|_1 = \sum_{i=1}^n |v_i| \qquad \|\mathbf{v}\|_\infty = \max_i |v_i|$$

- The 0-norm is a pseudo-norm as it does not satisfy condition 2

$$\|\mathbf{v}\|_0 = \text{number of non-zero components}$$

Norms for Regularisation

- Why so many norms? Here is one place where the choice matters
- When we fit a model many different weight vectors $\mathbf{w}$ explain the training data equally well
- We usually prefer a “small” $\mathbf{w}$, so we minimise

$$\text{loss}(\mathbf{w}) + \eta \|\mathbf{w}\|$$

- Equivalently we minimise the loss subject to $\|\mathbf{w}\| \leq c$, confining $\mathbf{w}$ to a **ball** of whichever norm we chose
- Each norm means something different by “small”
- $\|\mathbf{w}\|_2$ small: no one component is large, the weight is spread out
- $\|\mathbf{w}\|_1$ small: a few large components are tolerated provided the rest are tiny
- $\|\mathbf{w}\|_\infty$ small: only the largest component is controlled
- **Choosing a norm is choosing what you mean by “small”**

Norms for Regularisation

- Why so many norms? Here is one place where the choice matters
- When we fit a model many different weight vectors $\mathbf{w}$ explain the training data equally well
- We usually prefer a “small” $\mathbf{w}$, so we minimise

$$\text{loss}(\mathbf{w}) + \eta \|\mathbf{w}\|$$

- Equivalently we minimise the loss subject to $\|\mathbf{w}\| \leq c$, confining $\mathbf{w}$ to a **ball** of whichever norm we chose
- Each norm means something different by “small”
- $\|\mathbf{w}\|_2$ small: no one component is large, the weight is spread out
- $\|\mathbf{w}\|_1$ small: a few large components are tolerated provided the rest are tiny
- $\|\mathbf{w}\|_\infty$ small: only the largest component is controlled
- **Choosing a norm is choosing what you mean by “small”**

Norms for Regularisation

- Why so many norms? Here is one place where the choice matters
- When we fit a model many different weight vectors $\mathbf{w}$ explain the training data equally well
- We usually prefer a “small” $\mathbf{w}$, so we minimise

$$\text{loss}(\mathbf{w}) + \eta \|\mathbf{w}\|$$

- Equivalently we minimise the loss subject to $\|\mathbf{w}\| \leq c$, confining $\mathbf{w}$ to a **ball** of whichever norm we chose
- Each norm means something different by “small”
- $\|\mathbf{w}\|_2$ small: no one component is large, the weight is spread out
- $\|\mathbf{w}\|_1$ small: a few large components are tolerated provided the rest are tiny
- $\|\mathbf{w}\|_\infty$ small: only the largest component is controlled
- **Choosing a norm is choosing what you mean by “small”**

Norms for Regularisation

- Why so many norms? Here is one place where the choice matters
- When we fit a model many different weight vectors $\mathbf{w}$ explain the training data equally well
- We usually prefer a “small” $\mathbf{w}$, so we minimise

$$\text{loss}(\mathbf{w}) + \eta \|\mathbf{w}\|$$

- Equivalently we minimise the loss subject to $\|\mathbf{w}\| \leq c$, confining $\mathbf{w}$ to a **ball** of whichever norm we chose
- Each norm means something different by “small”
- $\|\mathbf{w}\|_2$ small: no one component is large, the weight is spread out
- $\|\mathbf{w}\|_1$ small: a few large components are tolerated provided the rest are tiny
- $\|\mathbf{w}\|_\infty$ small: only the largest component is controlled
- **Choosing a norm is choosing what you mean by “small”**

Norms for Regularisation

- Why so many norms? Here is one place where the choice matters
- When we fit a model many different weight vectors $\mathbf{w}$ explain the training data equally well
- We usually prefer a “small” $\mathbf{w}$, so we minimise

$$\text{loss}(\mathbf{w}) + \eta \|\mathbf{w}\|$$

- Equivalently we minimise the loss subject to $\|\mathbf{w}\| \leq c$, confining $\mathbf{w}$ to a **ball** of whichever norm we chose
- Each norm means something different by “small”
- $\|\mathbf{w}\|_2$ small: no one component is large, the weight is spread out
- $\|\mathbf{w}\|_1$ small: a few large components are tolerated provided the rest are tiny
- $\|\mathbf{w}\|_\infty$ small: only the largest component is controlled
- **Choosing a norm is choosing what you mean by “small”**

Norms for Regularisation

- Why so many norms? Here is one place where the choice matters
- When we fit a model many different weight vectors $\mathbf{w}$ explain the training data equally well
- We usually prefer a “small” $\mathbf{w}$, so we minimise

$$\text{loss}(\mathbf{w}) + \eta \|\mathbf{w}\|$$

- Equivalently we minimise the loss subject to $\|\mathbf{w}\| \leq c$, confining $\mathbf{w}$ to a **ball** of whichever norm we chose
- Each norm means something different by “small”
- $\|\mathbf{w}\|_2$ small: no one component is large, the weight is spread out
- $\|\mathbf{w}\|_1$ small: a few large components are tolerated provided the rest are tiny
- $\|\mathbf{w}\|_\infty$ small: only the largest component is controlled
- **Choosing a norm is choosing what you mean by “small”**

Norms for Regularisation

- Why so many norms? Here is one place where the choice matters
- When we fit a model many different weight vectors $\mathbf{w}$ explain the training data equally well
- We usually prefer a “small” $\mathbf{w}$, so we minimise

$$\text{loss}(\mathbf{w}) + \eta \|\mathbf{w}\|$$

- Equivalently we minimise the loss subject to $\|\mathbf{w}\| \leq c$, confining $\mathbf{w}$ to a **ball** of whichever norm we chose
- Each norm means something different by “small”
- $\|\mathbf{w}\|_2$ small: no one component is large, the weight is spread out
- $\|\mathbf{w}\|_1$ small: a few large components are tolerated provided the rest are tiny
- $\|\mathbf{w}\|_\infty$ small: only the largest component is controlled
- **Choosing a norm is choosing what you mean by “small”**

Norms for Regularisation

- Why so many norms? Here is one place where the choice matters
- When we fit a model many different weight vectors $\mathbf{w}$ explain the training data equally well
- We usually prefer a “small” $\mathbf{w}$, so we minimise

$$\text{loss}(\mathbf{w}) + \eta \|\mathbf{w}\|$$

- Equivalently we minimise the loss subject to $\|\mathbf{w}\| \leq c$, confining $\mathbf{w}$ to a **ball** of whichever norm we chose
- Each norm means something different by “small”
- $\|\mathbf{w}\|_2$ small: no one component is large, the weight is spread out
- $\|\mathbf{w}\|_1$ small: a few large components are tolerated provided the rest are tiny
- $\|\mathbf{w}\|_\infty$ small: only the largest component is controlled
- **Choosing a norm is choosing what you mean by “small”**

Norms for Regularisation

- Why so many norms? Here is one place where the choice matters
- When we fit a model many different weight vectors $\mathbf{w}$ explain the training data equally well
- We usually prefer a “small” $\mathbf{w}$, so we minimise

$$\text{loss}(\mathbf{w}) + \eta \|\mathbf{w}\|$$

- Equivalently we minimise the loss subject to $\|\mathbf{w}\| \leq c$, confining $\mathbf{w}$ to a **ball** of whichever norm we chose
- Each norm means something different by “small”
- $\|\mathbf{w}\|_2$ small: no one component is large, the weight is spread out
- $\|\mathbf{w}\|_1$ small: a few large components are tolerated provided the rest are tiny
- $\|\mathbf{w}\|_\infty$ small: only the largest component is controlled
- **Choosing a norm is choosing what you mean by “small”**

Matrix Norms

- We can define norms for other objects
- The norm of a matrix encodes how large the mapping is
- The Frobenius norm is defined by

$$\|\mathbf{A}\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n |A_{ij}|^2}$$

- Many other norms exist including 1-norm, max-norm, etc.
- Some, but not all, norms satisfy the inequality

$$\|\mathbf{AB}\| \leq \|\mathbf{A}\| \times \|\mathbf{B}\|$$

Matrix Norms

- We can define norms for other objects
- The norm of a matrix encodes how large the mapping is
- The Frobenius norm is defined by

$$\|\mathbf{A}\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n |A_{ij}|^2}$$

- Many other norms exist including 1-norm, max-norm, etc.
- Some, but not all, norms satisfy the inequality

$$\|\mathbf{AB}\| \leq \|\mathbf{A}\| \times \|\mathbf{B}\|$$

Matrix Norms

- We can define norms for other objects
- The norm of a matrix encodes how large the mapping is
- The Frobenius norm is defined by

$$\|\mathbf{A}\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n |A_{ij}|^2}$$

- Many other norms exist including 1-norm, max-norm, etc.
- Some, but not all, norms satisfy the inequality

$$\|\mathbf{AB}\| \leq \|\mathbf{A}\| \times \|\mathbf{B}\|$$

Matrix Norms

- We can define norms for other objects
- The norm of a matrix encodes how large the mapping is
- The Frobenius norm is defined by

$$\|\mathbf{A}\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n |A_{ij}|^2}$$

- Many other norms exist including 1-norm, max-norm, etc.
- Some, but not all, norms satisfy the inequality

$$\|\mathbf{AB}\| \leq \|\mathbf{A}\| \times \|\mathbf{B}\|$$

Matrix Norms

- We can define norms for other objects
- The norm of a matrix encodes how large the mapping is
- The Frobenius norm is defined by

$$\|\mathbf{A}\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n |A_{ij}|^2}$$

- Many other norms exist including 1-norm, max-norm, etc.
- Some, but not all, norms satisfy the inequality

$$\|\mathbf{AB}\| \leq \|\mathbf{A}\| \times \|\mathbf{B}\|$$

Compatible Norms

- A vector and matrix norm are said to be compatible if

$$\|\mathbf{M}\mathbf{v}\|_b \leq \|\mathbf{M}\|_a \times \|\mathbf{v}\|_b$$

(Spectral and Euclidean norms are compatible)

- Norms provide quick ways to bound the maximum growth of a vector under a mapping induced by the matrix
- We will see that a measure of the sensitivity of a mapping is in terms of the ratio of its maximum effect to its minimum effect on a vector
- This is known as the **conditioning**, given by $\|\mathbf{M}\| \times \|\mathbf{M}^{-1}\|$

Compatible Norms

- A vector and matrix norm are said to be compatible if

$$\|\mathbf{M}\mathbf{v}\|_b \leq \|\mathbf{M}\|_a \times \|\mathbf{v}\|_b$$

(Spectral and Euclidean norms are compatible)

- Norms provide quick ways to bound the maximum growth of a vector under a mapping induced by the matrix
- We will see that a measure of the sensitivity of a mapping is in terms of the ratio of its maximum effect to its minimum effect on a vector
- This is known as the **conditioning**, given by $\|\mathbf{M}\| \times \|\mathbf{M}^{-1}\|$

Compatible Norms

- A vector and matrix norm are said to be compatible if

$$\|\mathbf{M}\mathbf{v}\|_b \leq \|\mathbf{M}\|_a \times \|\mathbf{v}\|_b$$

(Spectral and Euclidean norms are compatible)

- Norms provide quick ways to bound the maximum growth of a vector under a mapping induced by the matrix
- We will see that a measure of the sensitivity of a mapping is in terms of the ratio of its maximum effect to its minimum effect on a vector
- This is known as the **conditioning**, given by $\|\mathbf{M}\| \times \|\mathbf{M}^{-1}\|$

Compatible Norms

- A vector and matrix norm are said to be compatible if

$$\|\mathbf{M}\mathbf{v}\|_b \leq \|\mathbf{M}\|_a \times \|\mathbf{v}\|_b$$

(Spectral and Euclidean norms are compatible)

- Norms provide quick ways to bound the maximum growth of a vector under a mapping induced by the matrix
- We will see that a measure of the sensitivity of a mapping is in terms of the ratio of its maximum effect to its minimum effect on a vector
- This is known as the **conditioning**, given by $\|\mathbf{M}\| \times \|\mathbf{M}^{-1}\|$

Why Should You Care?

- Deep learning involves multiplying the input (which we can think of as a vector $\mathbf{x}$) by many layers
- In CNNs we have convolutional layers and dense layers
- The effect of applying these layers can be represented by a matrix multiplication $\mathbf{x}_n = \mathbf{L}_n \mathbf{x}_{n-1}$
- We also do other things like applying ReLUs or pooling that change the magnitude, $\|\mathbf{x}_n\|$, of our representation
- If you are developing new architectures you want $\|\mathbf{x}_n\|$ neither to blow up nor vanish
- This can be controlled by carefully choosing $\|\mathbf{L}_n\|$

Why Should You Care?

- Deep learning involves multiplying the input (which we can think of as a vector $\mathbf{x}$) by many layers
- In CNNs we have convolutional layers and dense layers
- The effect of applying these layers can be represented by a matrix multiplication $\mathbf{x}_n = \mathbf{L}_n \mathbf{x}_{n-1}$
- We also do other things like applying ReLUs or pooling that change the magnitude, $\|\mathbf{x}_n\|$, of our representation
- If you are developing new architectures you want $\|\mathbf{x}_n\|$ neither to blow up nor vanish
- This can be controlled by carefully choosing $\|\mathbf{L}_n\|$

Why Should You Care?

- Deep learning involves multiplying the input (which we can think of as a vector $\mathbf{x}$) by many layers
- In CNNs we have convolutional layers and dense layers
- The effect of applying these layers can be represented by a matrix multiplication $\mathbf{x}_n = \mathbf{L}_n \mathbf{x}_{n-1}$
- We also do other things like applying ReLUs or pooling that change the magnitude, $\|\mathbf{x}_n\|$, of our representation
- If you are developing new architectures you want $\|\mathbf{x}_n\|$ neither to blow up nor vanish
- This can be controlled by carefully choosing $\|\mathbf{L}_n\|$

Why Should You Care?

- Deep learning involves multiplying the input (which we can think of as a vector $\mathbf{x}$) by many layers
- In CNNs we have convolutional layers and dense layers
- The effect of applying these layers can be represented by a matrix multiplication $\mathbf{x}_n = \mathbf{L}_n \mathbf{x}_{n-1}$
- We also do other things like applying ReLUs or pooling that change the magnitude, $\|\mathbf{x}_n\|$, of our representation
- If you are developing new architectures you want $\|\mathbf{x}_n\|$ neither to blow up nor vanish
- This can be controlled by carefully choosing $\|\mathbf{L}_n\|$

Why Should You Care?

- Deep learning involves multiplying the input (which we can think of as a vector $\mathbf{x}$) by many layers
- In CNNs we have convolutional layers and dense layers
- The effect of applying these layers can be represented by a matrix multiplication $\mathbf{x}_n = \mathbf{L}_n \mathbf{x}_{n-1}$
- We also do other things like applying ReLUs or pooling that change the magnitude, $\|\mathbf{x}_n\|$, of our representation
- If you are developing new architectures you want $\|\mathbf{x}_n\|$ neither to blow up nor vanish
- This can be controlled by carefully choosing $\|\mathbf{L}_n\|$

Why Should You Care?

- Deep learning involves multiplying the input (which we can think of as a vector $\mathbf{x}$) by many layers
- In CNNs we have convolutional layers and dense layers
- The effect of applying these layers can be represented by a matrix multiplication $\mathbf{x}_n = \mathbf{L}_n \mathbf{x}_{n-1}$
- We also do other things like applying ReLUs or pooling that change the magnitude, $\|\mathbf{x}_n\|$, of our representation
- If you are developing new architectures you want $\|\mathbf{x}_n\|$ neither to blow up nor vanish
- This can be controlled by carefully choosing $\|\mathbf{L}_n\|$

The Lipschitz Constant of a Network

- We can now put the Lipschitz condition and the matrix norm together
- Recall f is Lipschitz with constant K if

$$\|f(\mathbf{x}) - f(\mathbf{y})\| \leq K \|\mathbf{x} - \mathbf{y}\|$$

- If we compose two functions the constants **multiply**

$$K_{f \circ g} \leq K_f K_g$$

- A layer $\mathbf{x} \mapsto \mathbf{L}_n \mathbf{x}$ has $K = \|\mathbf{L}_n\|_2$, the **spectral norm**—this is just compatibility, $\|\mathbf{L}\mathbf{v}\| \leq \|\mathbf{L}\|_2 \|\mathbf{v}\|$
- ReLUs and max-pooling never stretch distances (though they can shrink them): $K = 1$
- A network is just a composition of layers, so

$$K_{\text{net}} \leq \prod_n \|\mathbf{L}_n\|_2$$

The Lipschitz Constant of a Network

- We can now put the Lipschitz condition and the matrix norm together
- Recall f is Lipschitz with constant K if

$$\|f(\mathbf{x}) - f(\mathbf{y})\| \leq K \|\mathbf{x} - \mathbf{y}\|$$

- If we compose two functions the constants **multiply**

$$K_{f \circ g} \leq K_f K_g$$

- A layer $\mathbf{x} \mapsto \mathbf{L}_n \mathbf{x}$ has $K = \|\mathbf{L}_n\|_2$, the **spectral norm**—this is just compatibility, $\|\mathbf{L}\mathbf{v}\| \leq \|\mathbf{L}\|_2 \|\mathbf{v}\|$
- ReLUs and max-pooling never stretch distances (though they can shrink them): $K = 1$
- A network is just a composition of layers, so

$$K_{\text{net}} \leq \prod_n \|\mathbf{L}_n\|_2$$

The Lipschitz Constant of a Network

- We can now put the Lipschitz condition and the matrix norm together
- Recall f is Lipschitz with constant K if

$$\|f(\mathbf{x}) - f(\mathbf{y})\| \leq K \|\mathbf{x} - \mathbf{y}\|$$

- If we compose two functions the constants **multiply**

$$K_{f \circ g} \leq K_f K_g$$

- A layer $\mathbf{x} \mapsto \mathbf{L}_n \mathbf{x}$ has $K = \|\mathbf{L}_n\|_2$, the **spectral norm**—this is just compatibility, $\|\mathbf{L}\mathbf{v}\| \leq \|\mathbf{L}\|_2 \|\mathbf{v}\|$
- ReLUs and max-pooling never stretch distances (though they can shrink them): $K = 1$
- A network is just a composition of layers, so

$$K_{\text{net}} \leq \prod_n \|\mathbf{L}_n\|_2$$

The Lipschitz Constant of a Network

- We can now put the Lipschitz condition and the matrix norm together
- Recall f is Lipschitz with constant K if

$$\|f(\mathbf{x}) - f(\mathbf{y})\| \leq K \|\mathbf{x} - \mathbf{y}\|$$

- If we compose two functions the constants **multiply**

$$K_{f \circ g} \leq K_f K_g$$

- A layer $\mathbf{x} \mapsto \mathbf{L}_n \mathbf{x}$ has $K = \|\mathbf{L}_n\|_2$, the **spectral norm**—this is just compatibility, $\|\mathbf{L}\mathbf{v}\| \leq \|\mathbf{L}\|_2 \|\mathbf{v}\|$
- ReLUs and max-pooling never stretch distances (though they can shrink them): $K = 1$
- A network is just a composition of layers, so

$$K_{\text{net}} \leq \prod_n \|\mathbf{L}_n\|_2$$

The Lipschitz Constant of a Network

- We can now put the Lipschitz condition and the matrix norm together
- Recall f is Lipschitz with constant K if

$$\|f(\mathbf{x}) - f(\mathbf{y})\| \leq K \|\mathbf{x} - \mathbf{y}\|$$

- If we compose two functions the constants **multiply**

$$K_{f \circ g} \leq K_f K_g$$

- A layer $\mathbf{x} \mapsto \mathbf{L}_n \mathbf{x}$ has $K = \|\mathbf{L}_n\|_2$, the **spectral norm**—this is just compatibility, $\|\mathbf{L}\mathbf{v}\| \leq \|\mathbf{L}\|_2 \|\mathbf{v}\|$
- ReLUs and max-pooling never stretch distances (though they can shrink them): $K = 1$
- A network is just a composition of layers, so

$$K_{\text{net}} \leq \prod_n \|\mathbf{L}_n\|_2$$

The Lipschitz Constant of a Network

- We can now put the Lipschitz condition and the matrix norm together
- Recall f is Lipschitz with constant K if

$$\|f(\mathbf{x}) - f(\mathbf{y})\| \leq K \|\mathbf{x} - \mathbf{y}\|$$

- If we compose two functions the constants **multiply**

$$K_{f \circ g} \leq K_f K_g$$

- A layer $\mathbf{x} \mapsto \mathbf{L}_n \mathbf{x}$ has $K = \|\mathbf{L}_n\|_2$, the **spectral norm**—this is just compatibility, $\|\mathbf{L}\mathbf{v}\| \leq \|\mathbf{L}\|_2 \|\mathbf{v}\|$
- ReLUs and max-pooling never stretch distances (though they can shrink them): $K = 1$
- A network is just a composition of layers, so

$$K_{\text{net}} \leq \prod_n \|\mathbf{L}_n\|_2$$

Why Care About K ?

- K tells us how much the output can move when we nudge the input

$$\|f(\mathbf{x} + \boldsymbol{\delta}) - f(\mathbf{x})\| \leq K \|\boldsymbol{\delta}\|$$

- **Adversarial examples** are tiny $\boldsymbol{\delta}$ that change the classification: a network with small K cannot be fooled this easily
- It is the same quantity that controls the signal: $K \gg 1$ and signals (and gradients) explode, $K \ll 1$ and they vanish
- Wasserstein GANs require the critic to have $K \leq 1$. One way to enforce this is **spectral normalisation**: divide each $\mathbf{L}_n$ by $\|\mathbf{L}_n\|_2$
- Bounds on how well a network generalises also depend on K
- But there is a trade-off: make K too small and the network is too smooth to fit the data

Why Care About K ?

- K tells us how much the output can move when we nudge the input

$$\|f(\mathbf{x} + \boldsymbol{\delta}) - f(\mathbf{x})\| \leq K \|\boldsymbol{\delta}\|$$

- **Adversarial examples** are tiny $\boldsymbol{\delta}$ that change the classification: a network with small K cannot be fooled this easily
- It is the same quantity that controls the signal: $K \gg 1$ and signals (and gradients) explode, $K \ll 1$ and they vanish
- Wasserstein GANs require the critic to have $K \leq 1$. One way to enforce this is **spectral normalisation**: divide each $\mathbf{L}_n$ by $\|\mathbf{L}_n\|_2$
- Bounds on how well a network generalises also depend on K
- But there is a trade-off: make K too small and the network is too smooth to fit the data

Why Care About K ?

- K tells us how much the output can move when we nudge the input

$$\|f(\mathbf{x} + \boldsymbol{\delta}) - f(\mathbf{x})\| \leq K \|\boldsymbol{\delta}\|$$

- **Adversarial examples** are tiny $\boldsymbol{\delta}$ that change the classification: a network with small K cannot be fooled this easily
- It is the same quantity that controls the signal: $K \gg 1$ and signals (and gradients) explode, $K \ll 1$ and they vanish
- Wasserstein GANs require the critic to have $K \leq 1$. One way to enforce this is **spectral normalisation**: divide each $\mathbf{L}_n$ by $\|\mathbf{L}_n\|_2$
- Bounds on how well a network generalises also depend on K
- But there is a trade-off: make K too small and the network is too smooth to fit the data

Why Care About K ?

- K tells us how much the output can move when we nudge the input

$$\|f(\mathbf{x} + \boldsymbol{\delta}) - f(\mathbf{x})\| \leq K \|\boldsymbol{\delta}\|$$

- **Adversarial examples** are tiny $\boldsymbol{\delta}$ that change the classification: a network with small K cannot be fooled this easily
- It is the same quantity that controls the signal: $K \gg 1$ and signals (and gradients) explode, $K \ll 1$ and they vanish
- Wasserstein GANs require the critic to have $K \leq 1$. One way to enforce this is **spectral normalisation**: divide each $\mathbf{L}_n$ by $\|\mathbf{L}_n\|_2$
- Bounds on how well a network generalises also depend on K
- But there is a trade-off: make K too small and the network is too smooth to fit the data

Why Care About K ?

- K tells us how much the output can move when we nudge the input

$$\|f(\mathbf{x} + \boldsymbol{\delta}) - f(\mathbf{x})\| \leq K \|\boldsymbol{\delta}\|$$

- **Adversarial examples** are tiny $\boldsymbol{\delta}$ that change the classification: a network with small K cannot be fooled this easily
- It is the same quantity that controls the signal: $K \gg 1$ and signals (and gradients) explode, $K \ll 1$ and they vanish
- Wasserstein GANs require the critic to have $K \leq 1$. One way to enforce this is **spectral normalisation**: divide each $\mathbf{L}_n$ by $\|\mathbf{L}_n\|_2$
- Bounds on how well a network generalises also depend on K
- But there is a trade-off: make K too small and the network is too smooth to fit the data

Why Care About K ?

- K tells us how much the output can move when we nudge the input

$$\|f(\mathbf{x} + \boldsymbol{\delta}) - f(\mathbf{x})\| \leq K \|\boldsymbol{\delta}\|$$

- **Adversarial examples** are tiny $\boldsymbol{\delta}$ that change the classification: a network with small K cannot be fooled this easily
- It is the same quantity that controls the signal: $K \gg 1$ and signals (and gradients) explode, $K \ll 1$ and they vanish
- Wasserstein GANs require the critic to have $K \leq 1$. One way to enforce this is **spectral normalisation**: divide each $\mathbf{L}_n$ by $\|\mathbf{L}_n\|_2$
- Bounds on how well a network generalises also depend on K
- But there is a trade-off: make K too small and the network is too smooth to fit the data

Function Norms

- Functions can also have norms, for example, if $f(x)$ is defined on some interval $\mathcal{I}$

$$\|f\|_{L^2} = \sqrt{\int_{x \in \mathcal{I}} f^2(x) dx}$$

- The L^2 vector space is the set of functions for which $\|f\|_{L^2} < \infty$
- The L^1 -norm is given by $\|f\|_{L^1} = \int_{x \in \mathcal{I}} |f(x)| dx$
- The infinite-norm is given by $\|f\|_{\infty} = \max_{x \in \mathcal{I}} |f(x)|$

Function Norms

- Functions can also have norms, for example, if $f(x)$ is defined on some interval $\mathcal{I}$

$$\|f\|_{L^2} = \sqrt{\int_{x \in \mathcal{I}} f^2(x) dx}$$

- The L^2 vector space is the set of functions for which $\|f\|_{L^2} < \infty$
- The L^1 -norm is given by $\|f\|_{L^1} = \int_{x \in \mathcal{I}} |f(x)| dx$
- The infinite-norm is given by $\|f\|_{\infty} = \max_{x \in \mathcal{I}} |f(x)|$

Function Norms

- Functions can also have norms, for example, if $f(x)$ is defined on some interval $\mathcal{I}$

$$\|f\|_{L^2} = \sqrt{\int_{x \in \mathcal{I}} f^2(x) dx}$$

- The L^2 vector space is the set of functions for which $\|f\|_{L^2} < \infty$
- The L^1 -norm is given by $\|f\|_{L^1} = \int_{x \in \mathcal{I}} |f(x)| dx$
- The infinite-norm is given by $\|f\|_{\infty} = \max_{x \in \mathcal{I}} |f(x)|$

Function Norms

- Functions can also have norms, for example, if $f(x)$ is defined on some interval $\mathcal{I}$

$$\|f\|_{L^2} = \sqrt{\int_{x \in \mathcal{I}} f^2(x) dx}$$

- The L^2 vector space is the set of functions for which $\|f\|_{L^2} < \infty$
- The L^1 -norm is given by $\|f\|_{L^1} = \int_{x \in \mathcal{I}} |f(x)| dx$
- The infinite-norm is given by $\|f\|_{\infty} = \max_{x \in \mathcal{I}} |f(x)|$

Norms and Metrics

- Given a norm, there exists an induced distance

$$d(\mathbf{x}, \mathbf{y}) = \|\mathbf{x} - \mathbf{y}\|$$

- Any normed space has an associated metric space
- Thus we can define the Frobenius distance between matrices

$$d_F(\mathbf{A}, \mathbf{B}) = \|\mathbf{A} - \mathbf{B}\|_F$$

- It is easy to see that non-negativity, symmetry and the triangular inequality for distances are all inherited from the same properties of norms

Norms and Metrics

- Given a norm, there exists an induced distance

$$d(\mathbf{x}, \mathbf{y}) = \|\mathbf{x} - \mathbf{y}\|$$

- Any normed space has an associated metric space
- Thus we can define the Frobenius distance between matrices

$$d_F(\mathbf{A}, \mathbf{B}) = \|\mathbf{A} - \mathbf{B}\|_F$$

- It is easy to see that non-negativity, symmetry and the triangular inequality for distances are all inherited from the same properties of norms

Norms and Metrics

- Given a norm, there exists an induced distance

$$d(\mathbf{x}, \mathbf{y}) = \|\mathbf{x} - \mathbf{y}\|$$

- Any normed space has an associated metric space
- Thus we can define the Frobenius distance between matrices

$$d_F(\mathbf{A}, \mathbf{B}) = \|\mathbf{A} - \mathbf{B}\|_F$$

- It is easy to see that non-negativity, symmetry and the triangular inequality for distances are all inherited from the same properties of norms

Norms and Metrics

- Given a norm, there exists an induced distance

$$d(\mathbf{x}, \mathbf{y}) = \|\mathbf{x} - \mathbf{y}\|$$

- Any normed space has an associated metric space
- Thus we can define the Frobenius distance between matrices

$$d_F(\mathbf{A}, \mathbf{B}) = \|\mathbf{A} - \mathbf{B}\|_F$$

- It is easy to see that non-negativity, symmetry and the triangular inequality for distances are all inherited from the same properties of norms

Summary

- Vector spaces with a distance (metric spaces) and vector spaces with a norm (normed vector spaces) are interesting objects
- They allow you to define a topology (open/closed sets, etc.)
- You can build up ideas about connectedness, continuity, contractive maps, fixed-point theorems, . . .
- For the most part we are going to consider an even more powerful vector space that has an inner-product defined

Summary

- Vector spaces with a distance (metric spaces) and vector spaces with a norm (normed vector spaces) are interesting objects
- They allow you to define a topology (open/closed sets, etc.)
- You can build up ideas about connectedness, continuity, contractive maps, fixed-point theorems, . . .
- For the most part we are going to consider an even more powerful vector space that has an inner-product defined

Summary

- Vector spaces with a distance (metric spaces) and vector spaces with a norm (normed vector spaces) are interesting objects
- They allow you to define a topology (open/closed sets, etc.)
- You can build up ideas about connectedness, continuity, contractive maps, fixed-point theorems, . . .
- For the most part we are going to consider an even more powerful vector space that has an inner-product defined

Summary

- Vector spaces with a distance (metric spaces) and vector spaces with a norm (normed vector spaces) are interesting objects
- They allow you to define a topology (open/closed sets, etc.)
- You can build up ideas about connectedness, continuity, contractive maps, fixed-point theorems, . . .
- For the most part we are going to consider an even more powerful vector space that has an inner-product defined